Numerical Analysis Final Project

B12502027 CHENG-HSUAN LEE

1.

(a) .

Two straight lines: $\begin{cases} L_1: \frac{x-5}{3} = \frac{y+7}{-6} = \frac{z-1}{-2} \\ L_2: \frac{x-1}{3} = \frac{y}{2} = \frac{z+5}{2} \end{cases}$

In addition, s is the parameter of the straight line, so the point P located on the straight line can be expressed as $L_1 \rightarrow L_1: \frac{x-5}{3} = \frac{y+7}{-6} = \frac{z-1}{-2} = sL_1P(s) = (P_x, P_y, P_z) = (3s+5, -6s-7, -2s+1)$

In addition, t is the parameter of the straight line, so the point Q located on the straight line can be expressed as $L_2 \rightarrow L_2: \frac{x-1}{3} = \frac{y}{2} = \frac{z+5}{2} = tL_2Q(t) = (Q_x, Q_y, Q_z) = (3t+1, 2t, 2t-5)$

Here we take the distance or the square of the distance as the cost function, but since we will need to use the position where the differential is equal to zero to find the minimum distance point (it can be imagined that this formula has no maximum value, so there will only be a point where the differential is equal to zero, this point is the minimum distance), so here we use the square of the distance without a root sign as the cost function to facilitate subsequent differentiation. In addition, the minimum distance and the minimum distance square should be at the same location, so using distance square is a feasible method.

$\overline{PQ} = \|P(s) - Q(t)\|_2$

$= \sqrt{(3s-3t+4)^2 + (-6s-2t-7)^2 + (-2s-2t+6)^2}$

$= \sqrt{49s^2 + 17t^2 + 14st + 84s - 20t + 101}$,

Another $cost\ function, J(s,t) = \overline{PQ}^2 = \|P(s) - Q(t)\|_2^2$

$= (3s-3t+4)^2 + (-6s-2t-7)^2 + (-2s-2t+6)^2$

$= 49s^2 + 17t^2 + 14st + 84s - 20t + 101$

(b) .

Observation shows that it is the sum of several squares, similar to a multi-variable notched upward bowl-shaped surface (the position where the differential is equal to zero can be excluded as the maximum value or saddle point), so by performing partial differentials on s and t respectively and substituting the parameters when both are equal to zero and the position expression, the minimum distance position can be obtained.$cost\ function\ J(s,t)P(s)Q(t)$

$\begin{cases}\frac{\partial F(s,t)}{\partial s}=98s+14t+84=0\\ \frac{\partial F(s,t)}{\partial t}=14s+34t-20=0\end{cases}\rightarrow\begin{cases}7s+t=-6\\ 7s+17t=10\end{cases}$, got ,$\begin{cases}s=-1\\ t=1\end{cases}$

Substituting back to the original position formula, when the distance between two points is the minimum value,$\begin{cases}s=-1\\ t=1\end{cases}\begin{cases}P(s)=(3s+5,-6s-7,-2s+1)\\ Q(t)=(3t+1,2t,2t-5)\end{cases}\rightarrow$

$\begin{cases}P(-1)=(2,-1,3)\\ Q(1)=(4,2,-3)\end{cases}cost=\|P(-1)-Q(1)\|_2^2=(-2)^2+(-3)^2+6^2=49$

shortest distance.$\overline{PQ}=\|P(-1)-Q(1)\|_2=\sqrt{49}=7$

(c) .

If it is assumed that point P and point Q coincide, it must satisfy , that is, the three coordinates are equal to respectively. For the convenience of observation, it can be written in matrix form and can be expressed as$P(s)=Q(t)\begin{cases}3s+5=3t+1\\ -6s-7=2t\\ -2s+1=2t-5\end{cases}\rightarrow\begin{cases}3s-3t=-4\\ -6s-2t=7\\ -2s-2t=-6\end{cases}\begin{bmatrix}3&-3\\ -6&-2\\ -2&-2\end{bmatrix}\begin{bmatrix}s\\ t\end{bmatrix}=\begin{bmatrix}-4\\ 7\\ -6\end{bmatrix}Au=b$, $A=\begin{bmatrix}3&-3\\ -6&-2\\ -2&-2\end{bmatrix}$,

$u=\begin{bmatrix}s\\ t\end{bmatrix},b=\begin{bmatrix}-4\\ 7\\ -6\end{bmatrix}$。

If a set of parameters can be found such that the above holds, then it can be determined that the two straight lines will intersect. However, if we look at this equation, we can see that there are only two unknowns in the question but

there are three equations that are not equal to each other. Therefore, we can judge that it is an overdetermined system. In this case, the system may not have an exact solution. After calculation, it can be found that this problem does not have an exact solution, that is, the two straight lines will not intersect, and there is no solution if point P and point Q coincide.$u = \begin{bmatrix} s \\ t \end{bmatrix} Au = bAu = b$

(d) .

If an exact solution to an equation cannot be obtained because the number of equations is greater than the number of unknowns, left pseudo inverse can be used to obtain the most ideal approximate solution with minimum square error:

Since cannot be established, we can make its residual value . At this time, the smaller the value, the smaller the parameter. The solution will be closer to the approximate solution we want. Following the concept of cost function above, let , since this formula is a sum of squares function, the minimum value will appear when the differential is equal to zero. Differentiate with respect to and make it equal to zero. After moving the terms, we can get the normal equation:, which is the left pseudo inverse, and the least square error solution is .$Au = bresidue, r = Au - b|r|||Au - b||^2 u = \begin{bmatrix} s \\ t \end{bmatrix} J(u) = (Au -$

$b)^T (Au - b) J(u) \frac{\partial J(u)}{\partial u} = \frac{\partial}{\partial u}\left((Au - b)^T (Au - b)\right) = 2A^T (Au - b) =$

$0 A^T Au = A^T b \rightarrow u = (A^T A)^{-1} A^T b (A^T A)^{-1} A^T u = (A^T A)^{-1} A^T b$

Topic above:$A = \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix}, u = \begin{bmatrix} s \\ t \end{bmatrix}, b = \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix}$

$A^T = \begin{bmatrix} 3 & -6 & -2 \\ -3 & -2 & -2 \end{bmatrix}, \quad A^T A = \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix}, \quad A^T b = \begin{bmatrix} -42 \\ 10 \end{bmatrix}$

$\rightarrow \begin{bmatrix} 49 & 7 \\ 7 & 17 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} = \begin{bmatrix} -42 \\ 10 \end{bmatrix} \rightarrow \begin{cases} s = -1 \\ t = 1 \end{cases}$

At this time, its minimum residual value =, the distance is, and its shortest distance.$r = Au - b \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} - \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix} = \begin{bmatrix} -2 \\ -3 \\ 6 \end{bmatrix} \|Au - b\|_2 = 7$

(e) .

In Q1(a) and Q1(b), we directly write the parameter formula to get and , directly write the function of the distance between two points and find its minimum value for the square of the distance. The purpose is that if we can minimize this formula, we can get the closest solution. In Q1(c) and Q1(d), from the above explanation, we can know that in order to obtain the most ideal approximate solution, we hope to minimize the value of the residual value , that is, to minimize$P - Q = (3s - 3t + 4, -6s - 2t - 7, -2s - 2t + 6) cost\ function, J(s,t) = \|P(s) - Q(t)\|_2^2 = (3s - 3t + 4)^2 + (-6s - 2t - 7)^2 + (-2s - 2t + 6)^2 r = Au - b\|Au - b\|^2 \|Au - b\|^2 =$

$$\left\| \begin{bmatrix} 3 & -3 \\ -6 & -2 \\ -2 & -2 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} - \begin{bmatrix} -4 \\ 7 \\ -6 \end{bmatrix} \right\| = (3s - 3t + 4)^2 + (-6s - 2t - 7)^2 +$$

$(-2s - 2t + 6)^2$

2.

(a) .

$y'' + 4y' + 4y = x, \quad y(0) = 0, \quad y(1) = 2$

Homogeneous Solution:, its Characteristic equation is , it can be obtained that is a multiple root solution, so its Homogeneous Solution is . $y'' + 4y' + 4y = 0 r^2 + 4r + 4 = 0 \rightarrow (r+2)^2 = 0 r = -2 y_h = C_1 e^{-2x} + C_2 x e^{-2x}$

Particular Solution: Since the right side of the equal sign is a linear term, we can assume that the Particular Solution is , and substitute it back into the original formula to get , so its Particular Solution is . $x y_p = ax + b, \quad y_p' = a, \quad y_p'' = 0 \rightarrow 0 + 4a + 4ax + 4b = x \begin{cases} a = \frac{1}{4} \\ b = -\frac{1}{4} \end{cases} y_p = \frac{1}{4}x - \frac{1}{4}$

General Solution: General Solution is the linear combination of Homogeneous Solution and Particular Solution. Substituting the question condition back into General Solution can get , that is, General Solution is . $y = y_h + y_p = C_1 e^{-2x} + C_2 x e^{-2x} + \frac{1}{4}x - \frac{1}{4} y(0) = 0, \quad y(1) =$

$2 \begin{cases} y(0) = C_1 - \frac{1}{4} = 0 \\ y(1) = C_1 e^{-2} + C_2 e^{-2} + \frac{1}{4} - \frac{1}{4} = 2 \end{cases} \rightarrow \begin{cases} C_1 = \frac{1}{4} \\ C_2 = 2e^2 - \frac{1}{4} \end{cases} y(x) = \frac{1}{4} e^{-2x} +$

$(2e^2 - \frac{1}{4}) x e^{-2x} + \frac{1}{4}x - \frac{1}{4}$

(b) .

(i).

We first need to define a uniform grid (the N value below is the total number of grid points we have taken in this grid), and observing the question conditions, we can find that they are all function values on the directly given boundary (Dirichlet boundary condition), which are the boundary points at both ends, and all points can be expressed as . According to the central difference formula (using the data on both sides of a point to approximate the derivative of this point), it can be seen that for a point, its second-order derivative can be approximated as , and its first-order derivative can be approximated as , substitute the two equations back into the original function,

and after simplifying the transfer terms, the discretization result can be obtained$x_0 = 0, \quad x_{N-1} = 1 x_i = i\Delta x, \quad i = 0, 1, 2, \cdots, N-1 \; with \; \Delta x = \frac{1}{N-1} x_i y''(x_i) \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2}, \quad i = 1, 2, \cdots, N-2 y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2\Delta x}, \quad i = 1, 2, \cdots, N-2 \rightarrow \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta x^2} + 4\frac{y_{i+1} - y_{i-1}}{2\Delta x} + 4y_i = x_i y_{i-1} - 2y_i + y_{i+1} + 2\Delta x y_{i+1} - 2\Delta x y_{i-1} + 4\Delta x^2 y_i = \Delta x^2 x_i$

$\rightarrow \left(\frac{1}{\Delta x^2} - \frac{2}{\Delta x}\right) y_{i-1} + \left(4 - \frac{2}{\Delta x^2}\right) y_i + \left(\frac{1}{\Delta x^2} + \frac{2}{\Delta x}\right) y_{i+1} = x_i$, and can be expressed as $.a y_{i-1} + b y_i + c y_{i+1} = x_i, \quad a = \left(\frac{1}{\Delta x^2} - \frac{2}{\Delta x}\right), b = \left(4 - \frac{2}{\Delta x^2}\right), c = \left(\frac{1}{\Delta x^2} + \frac{2}{\Delta x}\right)$

Finally, after substituting the boundary value, we can get that the first column (i.e.) in the matrix form is , the last column (i.e.) is , and each item in the middle term (i.e.) is . After writing the multiple equations obtained above into matrix form, we can get the linear equations as $.y_0 = 0, \quad y_{N-1} = 2i = 1 b y_1 + c y_2 = x_1 - a y_0 = \; x_1 i = N - 2 a y_{N-3} + b y_{N-2} = x_{N-2} - c y_{N-1} = \; x_{N-2} - 2ci = 2, 3, 4, \cdots, N-1 a y_{i-1} + b y_i + c y_{i+1} = x_i Ay =$

$$d \begin{bmatrix} b & c & 0 & & 0 & 0 & 0 \\ a & b & c & \cdots & 0 & 0 & 0 \\ 0 & a & b & & 0 & 0 & 0 \\ & \vdots & & \ddots & & \vdots & \\ 0 & 0 & 0 & & b & c & 0 \\ 0 & 0 & 0 & \cdots & a & b & c \\ 0 & 0 & 0 & & 0 & a & b \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-4} \\ y_{N-3} \\ y_{N-2} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-4} \\ x_{N-3} \\ x_{N-2} - 2c \end{bmatrix}$$

(ii) & (iii).

Gaussian Elimination is used here to solve this linear system of equations: The principle of Gaussian Elimination is that after we obtain a system of equations , we use column-to-column operation elimination and simplification to simplify the matrix into a relatively simple upper triangular form, and then start calculations from the last column and deduce all the previous solutions step by step. When using Gaussian Elimination here, partial pivoting is used. Before performing the elimination operation, we will first find the number with the largest absolute value in this column as the pivot. If necessary, we will exchange the two columns to avoid dividing by 0 or dividing by too small

a number, which can increase the stability of the calculation. And why I use Gaussian Elimination here I will explain later.$Ay = dAy$

Here I use the MATLAB program to help me calculate the matrix using Gaussian Elimination, and set as a variable and ask the program to output , Maximum Error (tables and relationship diagrams to help me observe the performance of the error under different conditions.$\Delta x \Delta x MAX(y_{Analysis} - y_{Numerical})\Delta x$

Table 1. Maximum Error corresponding table (Gaussian Elimination) when differentΔx

Δx	Max Error Value
0.25	0.21972
0.125	0.048279
0.0625	0.011721
0.03125	0.0029124
0.015625	0.00072788
0.0078125	0.00018189
0.0039062	4.5468e-05
0.0019531	1.1367e-05

Here we will set to 1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512, and list the maximum error values under different conditions to help me observe the error.$\Delta x \Delta x$

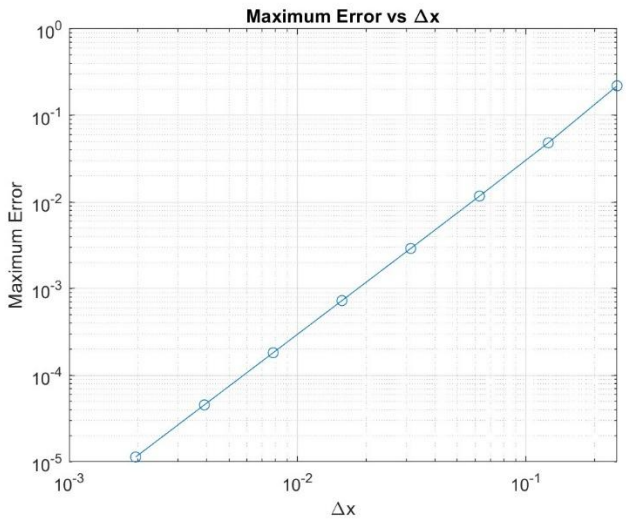


Figure 1. Maximum Error and Gaussian Elimination diagramΔx

It can be seen from the above figure that when the grid is cut into finer parts, the error between the numerical solution and the analytical solution we obtain will be smaller, and since the relationship between the two is close to a straight line with a slope of 2 in the log-log graph, it can be inferred that the relationship between the two is quadratic, that is, every time when becomes 1/2 of the original, the Maximum Error will drop to 1/4 of the original. The conclusion drawn here is the same as that of the expectation theory. Since we used the second-order central difference method in the discretization of this question, theoretically the error should be when using the central difference method to approximate the first-order and second-order derivatives. This also allows us to see that the finite difference method used here has second-order convergence characteristics.$\Delta x O(\Delta x^2)$

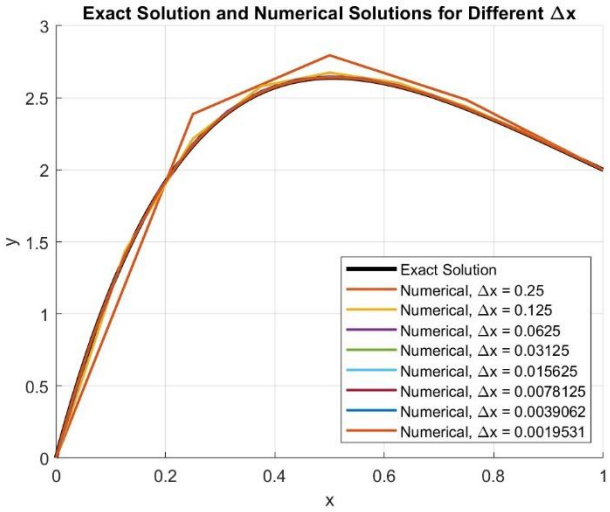


Figure 2. Comparison of analytical solutions and numerical solutions under different conditions (Gaussian Elimination)Δx

Table 2. Maximum error point and error proportion corresponding table (Gaussian Elimination) at different timesΔx

Δx	The biggest error	Correct solutiony_{exact}	Numerical solution$y_{numerical}$	error ratio
1/4	$x = 0.25$	2.1671	2.3868	10.14%
1/8	x =0.25	2.1671	2.2153	2.23%
1/16	x =0.25	2.1671	2.1788	0.54%
1/32	x =0.28125	2.2909	2.2938	0.13%

1/64	x =0.26563	2.2320	2.2327	0.033%
1/128	x =0.26563	2.2320	2.2322	0.0081%
1/256	x =0.26563	2.2320	2.2320	0.0020%
1/512	x =0.26563	2.2320	2.2320	0.0005%

In Figure 2 here, we compare the analytical solution and the numerical solution under different conditions. We can find that if we simply observe it with the naked eye, due to its second-order convergence characteristics, although the error is still very large and the deviation of the analytical solution is very large, when is halved, the error instantly drops to 1/4 of the original. It can also be found in the figure that it is obviously close to the analytical solution, and when is halved again (i.e.) the error has been reduced to the original 1/16, in the figure it almost coincides with the analytical solution, and in the subsequent finer mesh, the maximum error is almost less than . Compared with the analytical solution value at the maximum error, it is nearly three orders of magnitude smaller. It can be observed that the convergence speed is very fast.$\Delta x\Delta x = 1/4\Delta x\Delta x\Delta x = 1/1610^{-3}$

Observing the data compiled in Table 2, we can find that when the maximum error is halved, the error drops by about 1/4. When the grid is cut to , the error is already lower than , and it can be found that it drops quickly.$\Delta x\Delta x = 1/161\%$

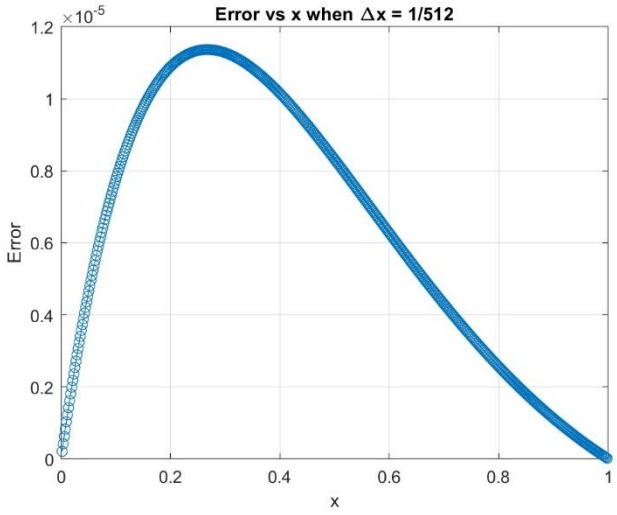


Figure 3. Error absolute value distribution (Gaussian Elimination) at that time$\Delta x = 1/512$

Here we specifically discuss the thinnest point cuts and the smallest errors. It

can be observed that on both sides of the boundary, the value is not approximated, but given directly, so the error will be equal to 0, and the error will be larger toward the middle. The maximum value falls at (although this position is the maximum error, but it is only , which is still about times less than the analytical solution 2.2397), and the overall trend is to shift toward . It is speculated that the reason is related to the structure of the overall ODE. Since there are terms in this equation, this first-order derivative term will cause the overall equation to have a similar offset effect (from Figure 2 above, it can also be seen that this equation obviously changes significantly when it is close to), and the magnitude of the change in the equation will in turn affect the distribution of the error. In addition, it can also be observed that when we write the analytical solution , it contains terms. This change is more obvious when approaching , and the process of moving toward will gradually attenuate. This behavior makes the overall function change more strongly in the left area, thus making errors in this area easy to occur and amplify.$\Delta x = 1/512 x = 0,\quad x = 1 x = 0.265 \sim 0.269\ 1.1367 \times 10^{-5} 10^{5} x = 0 4 y' x = 0 x = 1 y(x) = \frac{1}{4} e^{-2x} + (2e^{2} - \frac{1}{4}) x e^{-2x} + \frac{1}{4} x - \frac{1}{4} e^{-2x} e^{-2x} x = 0 x = 1$

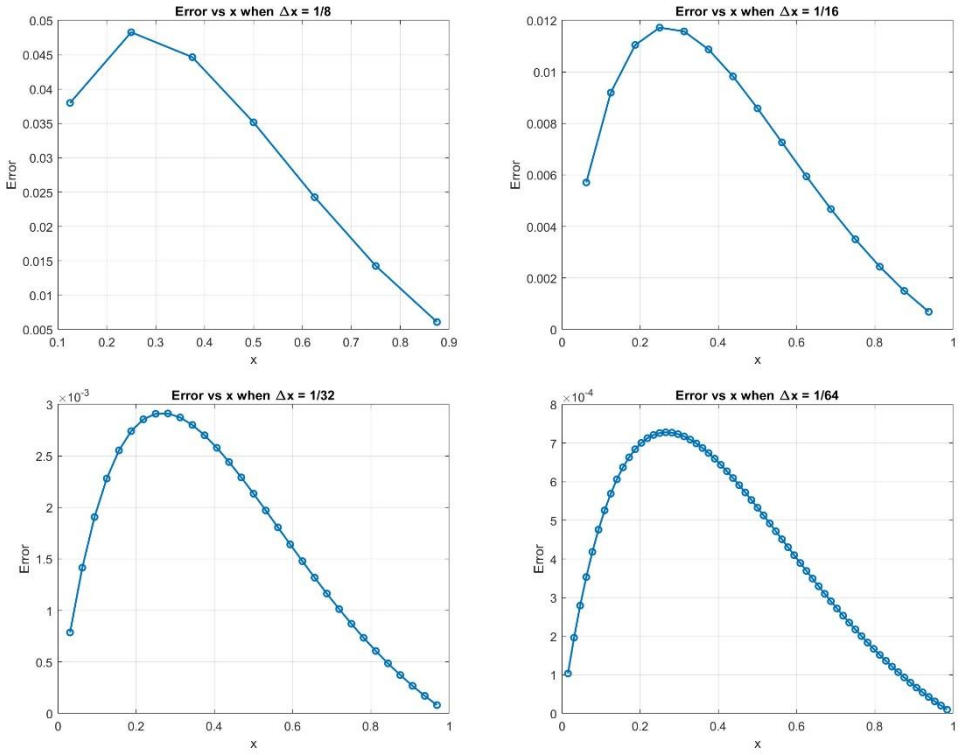


Figure 4/Figure 5/Figure 6/Figure 7. Error absolute value distribution

(Gaussian Elimination) at different timesΔx

Subsequently, for the smaller output error absolute value distribution, it can be found that even if is not as large as the previous example, the error distribution pattern is still about the same length, and is a bell-shaped curve shifted toward , with a large difference only in the absolute value of the vertical axis error. This phenomenon can then be inferred that this error distribution is a structural characteristic of the overall ODE, and has little to do with the detail of the grid cut.$\Delta x\Delta xx = 0$

Comparison of Gaussian Elimination and other methods for calculating linear equations: There are four main methods mentioned in the professor's presentation, namely Gauss elimination, LU decomposition, Jacobi iteration, and Gauss-Seidel iteration. From the calculation process, we can know that the conclusion we got above - the fact that Maximum Error and has a quadratic relationship has nothing to do with the method we choose, only the process of discretizing the original equation. If we use the second-order central difference method here, then no matter which method we use to calculate the matrix taught by the professor, the conclusion we get should be the same. The reason why we use Gaussian Elimination here is because this method can intuitively and directly solve a more accurate solution at one time, without having to go through multiple iterations like Jacobi iteration and Gauss-Seidel iteration to approximate the solution we want, which is cumbersome and inaccurate. LU decomposition is more suitable for using the same group to solve multiple groups of . It is not very helpful for the operations required for this problem, so I finally chose Gauss elimination, which is the most intuitive and simple to understand. However, Gauss elimination also has its shortcomings. For a matrix, the amount of operations using Gauss elimination is approximately , which means that if the unknown number becomes 2 times, the amount of operations becomes 8 times. Therefore, when the grid is cut very finely, that is, when the value is very large (for example,) and the matrix has many entries, the amount of operations of this method will become very large. However, we only use here, so there will be no problem of computer lag due to the amount of operations.$\Delta xAdn \times nO(n^3)NN > 1000N = 513$

(c) .

(i).

Since the conditions given in this question fall on the boundaries of and , it can be identified as a BVP. However, many numerical methods are better at solving IVPs, so here we use the shooting method to split this BVP into two IVPs, and use the prescribed numerical methods to solve this IVPs.$x = 0x =$ 1

Shooting method concept: Since we want to force BVP to IVP, however, we only know here and unknown. We first assume an initial slope satisfies the two conditions required to solve IVP, and then solve all the way to the right starting from$y(0) = 0y'(0) = 0y'(0) = \alpha \begin{cases} y(0) = 0 \\ y'(0) = \alpha \end{cases} x = 0x = 1\alpha x =$ $1\alpha x = 1\alpha x = 1y(1) = 2$

In this question, because the equation is linear, we actually don’ t need to guess randomly as mentioned above. We can directly split it into two IVPs: let be the solution of non-homogeneous IVP;$\alpha uvu + \theta v\theta y(1) = 2\theta$

Original function:$y'' + 4y' + 4y = x, \quad y(0) = 0, \quad y(1) = 2$

Assume , the equation can be simplified to . Since this is a linear equation, we can directly decompose it into two IVPs, let be the solution of non-homogeneous IVP,,,; let be the solution of homogeneous IVP,,,. From the linear superposition characteristics, it can be seen that the complete solution of IVP is . Substituting the right boundary condition (i.e.) into , we can derive that .$w(x) = \begin{bmatrix} y(x) \\ y'(x) \end{bmatrix} \frac{dw}{dx} = Aw + B = \begin{bmatrix} 0 & 1 \\ -4 & -4 \end{bmatrix} \begin{bmatrix} y(x) \\ y'(x) \end{bmatrix} + \begin{bmatrix} 0 \\ x \end{bmatrix} u_1 u(x) =$

$\begin{bmatrix} u_1 \\ u_1' \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \frac{du}{dx} = Au + Bu_1(0) = 0, \quad u_2(0) = 0 \rightarrow u(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} v_1 v(x) =$

$\begin{bmatrix} v_1 \\ v_1' \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} \frac{dv}{dx} = Avv_1(0) = 0, \quad v_2(0) = 1 \rightarrow v(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix} w = u +$

$\theta vy(1) = 2 \rightarrow u_1(1) + \theta v_1(1) = 2\theta = \frac{2 - u_1(1)}{v_1(1)} y(x) = u_1(x) +$

$\frac{2 - u_1(1)}{v_1(1)} v_1(x)$

(ii).

Here we use the Explicit Euler Method (one-step method) to solve this IVPs.

This method is simply: the value of the next point = the current value + the step size × the current slope. Taking the forward difference as the basic concept, after derivation, we can get .$y'(t_n) = f(t_n, y_n) \approx \frac{y_{n+1}-y_n}{\Delta t} \rightarrow y_{n+1} = y_n + \Delta t f(t_n, y_n)$

Apply this method to the two IVPs just obtained (here we cut the grid into equal parts, with is the step size and the grid points):$N\Delta x x_n = n\Delta x, \quad n = 0, 1, 2, \cdots, N$

For the non-homogeneous solution, substituting the Explicit Euler Method gives .$u_1' = u_2, \quad u_2' = -4u_2 - 4u_1 +$

$x\begin{cases} u_{1,n+1} = u_{1,n} + \Delta x u_{2,n} \\ u_{2,n+1} = u_{2,n} + \Delta x(-4u_{2,n} - 4u_{1,n} + x) \end{cases}$

For the homogeneous solution, substituting the Explicit Euler Method gives .$v_1' = v_2, \quad v_2' = -4v_2 - 4v_1 \begin{cases} v_{1,n+1} = v_{1,n} + \Delta x v_{2,n} \\ v_{2,n+1} = v_{2,n} + \Delta x(-4v_{2,n} - 4v_{1,n}) \end{cases}$

It can be seen from the above derivation that after substituting, we can get , where will be related to and . When the grid is cut finer, the error of and obtained when calculating from to step by step will be smaller, and will be more accurate. We can finally express the solution as .$\theta = \frac{2-u_1(1)}{v_1(1)}, \quad u_1(1) = u_{1,N}, \quad v_1(1) = v_{1,N}\theta_{\Delta x} = \frac{2-u_{1,N}}{v_{1,N}}\theta\Delta x x = 0x = 1u_{1,N}v_{1,N}\theta_{\Delta x} y_n = u_{1,n} + \frac{2-u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \cdots, N$

(iii).

Here I ask the MATLAB program to use Explicit Euler Method to assist in the calculation of this IVPs and plot different plots to facilitate my final error discussion.Δx

Table 3. Maximum Error corresponding table (Explicit Euler Method) when differentΔx

Δx	Max Error Value
0.25	1.7392
0.125	0.56023
0.0625	0.23937
0.03125	0.11182

0.015625	0.054072
0.0078125	0.026597
0.0039062	0.013192
0.0019531	0.0065691

Compared with the previous Gaussian Elimination, the cut-off points are set to 1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512, and the Max Error value is output.Δx

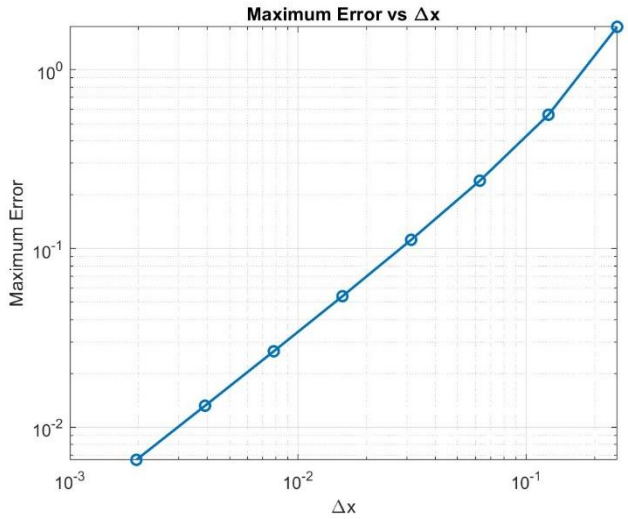


Figure 8. Maximum Error and relationship diagram (Explicit Euler Method)Δx

It can be observed here that in the method of using the Explicit Euler Method to calculate two IVPs and finally merging them into one BVP, the finer the grid is cut, the smaller the error, and the relationship between Maximum Error and will also approximate a straight line in the log-log plot. However, this is a straight line with a slope of 1. It can be inferred that the relationship between the two is a power, that is, every time when becomes 1/2 of the original, Maximum Error will also drop to 1/2 of the original. The conclusion drawn here is the same as that of the expected theory. In the Explicit Euler Method, its Truncation Error (assuming from to) is . If the Truncation Error from to is accumulated, it can be seen that the error of the Explicit Euler Method will theoretically be proportional to the first power. Here we can find that the conclusions we draw using MATLAB are consistent with the theory.$\Delta x\Delta x x_n x_{n+1} y_{n+1} - \left(y_n + \Delta x f(x_n, y_n)\right) = \sum_{k=0}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) -$

$$\left(y_n + \Delta x y'(x_n)\right) = \sum_{k=2}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) \sim O(\Delta x^2) x = 0 x = 1 \rightarrow$$

$$\frac{1}{\Delta x} \times O(\Delta x^2) \sim O(\Delta x) \Delta x$$

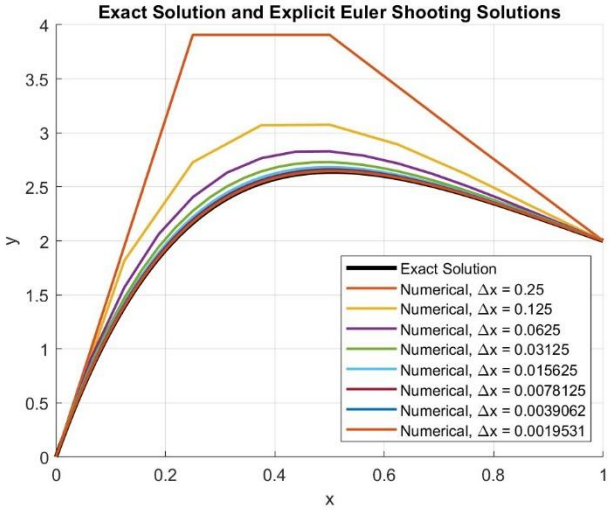


Figure 9. Comparison of analytical solutions and numerical solutions under different conditions (Explicit Euler Method)Δx

Table 4. Maximum error point and error proportion corresponding table at different times (Explicit Euler Method)Δx

Δx	The biggest error	Correct solutiony_{exact}	Numerical solution$y_{numerical}$	error ratio
1/4	$x = 0.25$	2.1671	3.9062	80.25%
1/8	x =0.25	2.1671	2.7273	25.85%
1/16	x =0.3125	2.3920	2.6314	10.01%
1/32	x =0.28125	2.2909	2.4027	4.88%
1/64	x =0.29688	2.3442	2.3982	2.31%
1/128	x =0.28906	2.3182	2.3448	1.15%
1/256	x =0.29297	2.3314	2.3446	0.57%
1/512	x =0.29297	2.3314	2.3379	0.28%

Here we compare the analytical solution and the numerical solution of the Explicit Euler Method under different . It can be seen from the above operation and the plot that this method has first-order convergence in error convergence. Therefore, when simply observing Figure 9, whenever halve, the error convergence speed will not be as obvious as the second-order

convergence of Gaussian Elimination.$\Delta x\Delta x$

Looking at Figure 9, we can see a special phenomenon, that is, the errors are positive and negative in the Gaussian Elimination above, but when it comes to the Explicit Euler Method, we can find that the errors are only positive deviations. I speculate that the reason here is due to the central difference method used in the discretization process of Gaussian Elimination. This method also refers to the grid point information on the left and right sides, so the error distribution will be more symmetrical. In some places, it will be greater than the analytical solution, and in some places it will be lower. However, the Explicit Euler Method is unidirectional. The concept is to calculate step by step from the left boundary all the way to the right. At each step, the slope of the current grid point is used to predict the next grid point. Therefore, the error will accumulate along the direction of calculation and show a deviation in a fixed direction. And because the slope of this function rises rapidly in the first half and only becomes flat in the middle, it is easy to overestimate when estimating the next grid using the old slope.
Observing Table 4, we can find that except for a few points where the calculation is very inaccurate due to very rough grid cuts (such as and) (the error even soars to when), the other errors decrease at a rate similar to that expected, roughly when is reduced to half of the original, the maximum error also drops to half of the original, and because its training speed is only first-order convergence, the error will not drop to less than 1% until when , and the convergence speed is much slower than Gaussian Elimination. In the future, I will also conduct further comparison and analysis of all numerical solutions.$\Delta x = 1/4\Delta x = 1/8\Delta x = 1/480.25\%\Delta x\Delta x = 1/256$

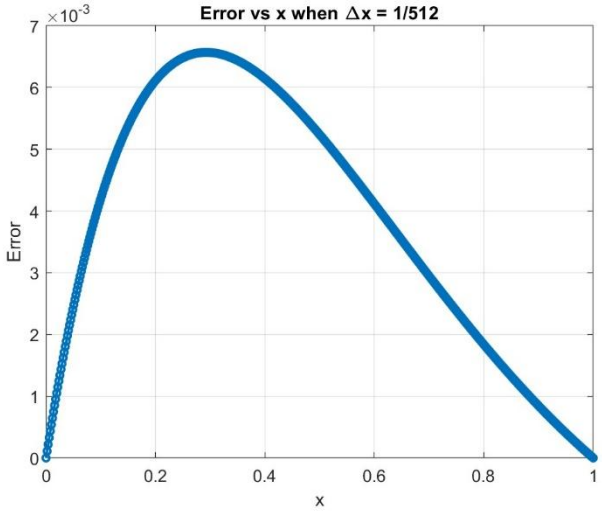


Figure 10. Error absolute value distribution at that time (Explicit Euler Method)$\Delta x = 1/512$

Here we also output the distribution of time error absolute value pairs with the finest grid. It is found that the error distribution is roughly the same as the distribution of Gaussian Elimination. They both show a biased bell-shaped curve, with only some obvious differences in the magnitude of the vertical axis. From the previous description of Q(ii)&(iii), it can be known that the distribution phenomenon of this error has nothing to do with the accuracy of the mesh cut. Another new conclusion that can be drawn from this question is that this phenomenon has nothing to do with the numerical method of calculation, but is only related to the structure of the equation itself.$\Delta x = 1/512xx = 0$

(iv).

The second time-marching scheme used here is the Crank-Nicolson Method (one-step method). The concept of this method is similar to the Explicit Euler Method used above, except that the concept becomes: next point value = current value + step size $\times$ average value of the slope of the current point and the next point. Its advantage compared to the Explicit Euler Method is that if the slope of a curve changes rapidly in a certain interval, the Explicit Euler Method will have obvious errors at this time, but due to the Crank-Nicolson method What is used is the average of the slope of the current point and the next point, so its comparison can reflect the change of the slope in this small interval, and the obtained value will be relatively more accurate. Since , the

relationship between and can be expressed as , if we use the trapezoidal method to approximate this integral at the integral (that is), where the height of the left end is , and the height of the right end is , substituting can get .$y' = f(t,y)\ y_n y_{n+1} y_{n+1} = y_n + \int_{t_n}^{t_n+\Delta t} f(t,y)dt\ integration \approx \Delta t \times (left - end\ height + right - end\ height)/2 f(t_n,y_n) f(t_{n+1},y_{n+1}) y_{n+1} = y_n + \frac{\Delta t}{2}(f(t_n,y_n) + f(t_{n+1},y_{n+1}))$

Apply this method to the two IVPs just obtained (here we cut the grid into equal parts, with is the step size and the grid points):$N\Delta x x_n = n\Delta x, \quad n = 0,1,2,\cdots,N$

Recall the past$\frac{dw}{dx} = Aw + B(x) = \begin{bmatrix} 0 & 1 \\ -4 & -4 \end{bmatrix}\begin{bmatrix} y(x) \\ y'(x) \end{bmatrix} + \begin{bmatrix} 0 \\ x \end{bmatrix}$

For the non-homogeneous solution, , substitute the Crank-Nicolson Method to get), and move backward to simplify to get , .$u(x) = \begin{bmatrix} u_1 \\ u_1' \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}, \quad u_1(0) = 0, \quad u_2(0) = 0 \ \rightarrow u(0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \frac{du}{dx} = Au + B(x) u_{n+1} = u_n + \frac{\Delta x}{2}(Au_n + B(x_n) + Au_{n+1} + B(x_{n+1}) \rightarrow \left[I - \frac{\Delta x}{2}A\right] u_{n+1} = \left[I + \frac{\Delta x}{2}A\right] u_n + \frac{\Delta x}{2}(B(x_n) + B(x_{n+1}))I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

For the homogeneous solution, , substitute the Crank-Nicolson Method to get), and move backward to simplify to get , .$v(x) = \begin{bmatrix} v_1 \\ v_1' \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}, \quad v_1(0) = 0, \quad v_2(0) = 1 \ \rightarrow v(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \frac{dv}{dx} = Avv_{n+1} = v_n + \frac{\Delta x}{2}(Av_n + Av_{n+1} \rightarrow \left[I - \frac{\Delta x}{2}A\right] v_{n+1} = \left[I + \frac{\Delta x}{2}A\right] v_n I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

order$M_1 = I - \frac{\Delta x}{2}A, \quad M_2 = I + \frac{\Delta x}{2}A$

$\rightarrow \begin{cases} M_1 u_{n+1} = M_2 u_n + \frac{\Delta x}{2}(B(x_n) + B(x_{n+1})) \\ M_1 v_{n+1} = M_2 v_n \end{cases}$, here and are two matrices.

This method calculates a linear system at each step to get and . By calculating the descendant return of , we can get the solution of BVP.$M_1 M_2 2 \times 2 u_{n+1} v_{n+1} \theta_{\Delta x} = \frac{2-u_{1,N}}{v_{1,N}} w = u + \theta v \rightarrow \ y_n = u_{1,n} + \frac{2-u_{1,N}}{v_{1,N}} v_{1,n}, \quad n =$

$0, 1, 2, \cdots, N$

Here I ask the MATLAB program to use the Crank-Nicolson Method to assist in the calculation of this IVPs and plot different plots to facilitate my final error discussion.Δx

Table 5. Maximum Error corresponding table (Crank-Nicolson Method) when differentΔx

Δx	Max Error Value
0.25	0.072746
0.125	0.017444
0.0625	0.004365
0.03125	0.0010903
0.015625	0.00027264
0.0078125	6.815e-05
0.0039062	1.7039e-05
0.0019531	4.2596e-06

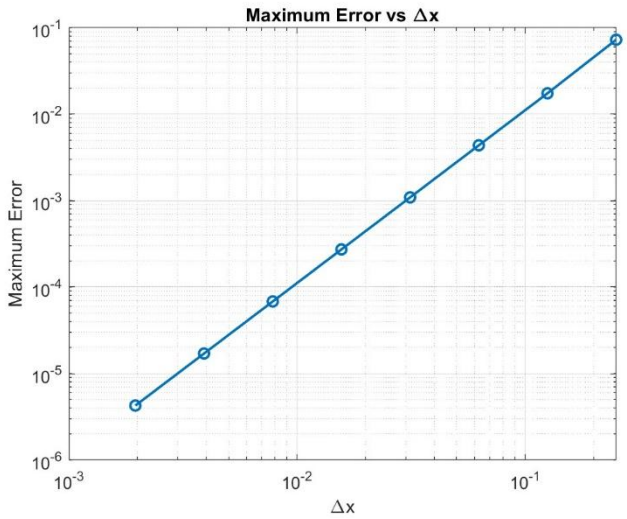


Figure 11. Maximum Error and relationship diagram (Crank-Nicolson Method)Δx

The trend is the same as before. The denser the grid, the smaller the maximum error will be. Looking at the above figure, we can find that in the Crank-Nicolson Method, the maximum error sum is approximately a straight line with a slope of 2 in the log-log diagram. It can be inferred that the relationship between the two is quadratic, that is, when the error becomes 1/2 of the original, the error will become 1/4 of the original. Theoretically speaking, the

Truncation Error of the Crank-Nicolson Method (assuming from to) is . If the Truncation Errors from to are accumulated, it can be known theoretically that the errors of the Crank-Nicolson Method are related to , which is consistent with the results we obtained after calculation and graphing at that time.$\Delta x\Delta x x_n x_{n+1} y_{n+1} - y_n - \frac{\Delta x}{2}\left(f(x_n, y_n) + f(x_{n+1}, y_{n+1})\right) =$

$\sum_{k=3}^{\infty} \frac{\Delta x^k}{k!} y^{(k)}(x_n) - \sum_{k=2}^{\infty} \frac{\Delta x^{k+1}}{2\times k!} y^{(k+1)}(x_n) = \sum_{k=3}^{\infty} (\frac{1}{k} -$

$\frac{1}{2}) \frac{\Delta x^k}{(k-1)!} y^{(k)}(x_n) \sim O(\Delta x^3) x = 0 x = 1 \rightarrow \frac{1}{\Delta x} \times O(\Delta x^3) \sim O(\Delta x^2) \Delta x^2$

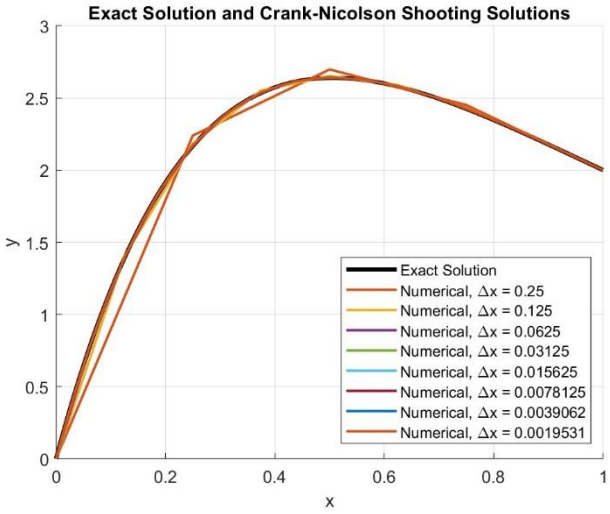


Figure 12. Comparison of analytical solutions and numerical solutions under different conditions (Crank-Nicolson Method)Δx

Table 6. Corresponding table of the maximum error point and error proportion at different times (Crank-Nicolson Method)Δx

Δx	The biggest error	Correct solutiony_{exact}	Numerical solution$y_{numerical}$	error ratio
1/4	$x = 0.25$	2.1671	2.2398	3.36%
1/8	x =0.25	2.1671	2.1845	0.80%
1/16	x =0.3125	2.3920	2.3964	0.18%
1/32	x =0.28125	2.2909	2.2920	0.048%
1/64	x =0.29688	2.3442	2.3444	0.012%
1/128	x =0.28906	2.3182	2.3183	0.0029%
1/256	x =0.29297	2.3314	2.3314	0.0007%
1/512	x =0.29297	2.3314	2.3314	0.0002%

Observing the error performance of the Crank-Nicolson Method, we can find that its calculation accuracy is much better than that of the Explicit Euler Method. When the grid accuracy is the worst (), the error is still only , and due to its second-order convergence characteristics, it can be observed in Figure 12 that its convergence speed is very fast. When , its calculated value roughly coincides with the analytical solution, and Table 6 can also see that its error convergence speed is about 1/4 The speed is converging, and when the grid is cut to , the error is already less than 1%.$\Delta x = 1/43.36\%\Delta x = 1/16\Delta x = 1/8$ In addition, observing Figure 12, we can find that the error is positive and negative like Gaussian Elimination. It is speculated that the reason is that unlike the Explicit Euler Method (which only uses one-sided prediction from left to right), the Explicit Euler Method will take both the left and right ends of the calculation point into consideration, that is, the average slope = (left end point slope + right end point slope) / 2, so its error will not show a fixed upward or fixed downward trend, and the overall error will be positive or negative.

Comparison and analysis of three methods (Gaussian Elimination, Explicit Euler Method, Crank-Nicolson Method):

Figure 13. Maximum Error and relationship diagram (comparison of three methods)Δx

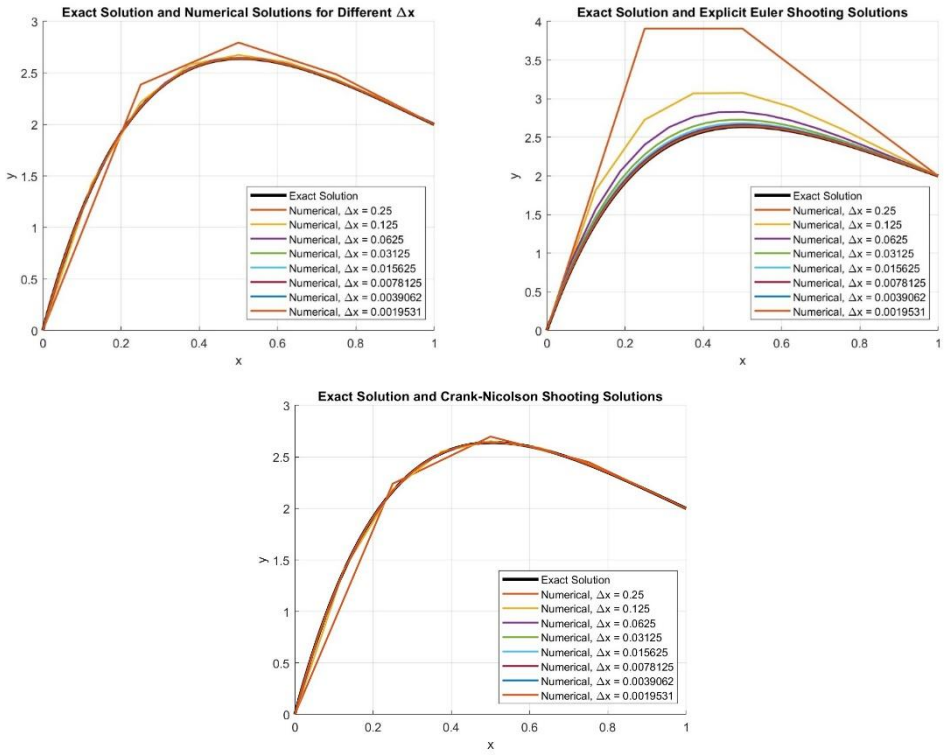


Figure 14/Figure 15/Figure 16, comparison chart of analytical solution and numerical solution under different conditions (comparison of three methods)Δx

Table 7. Corresponding table of error ratio at the maximum error point at different times (comparison of three methods)Δx

Δx	Gaussian Elimination error ratio	Explicit Euler Method error ratio	Crank–Nicolson Method error ratio
1/4	10.14%	80.25%	3.36%
1/8	2.23%	25.85%	0.80%
1/16	0.54%	10.01%	0.18%
1/32	0.13%	4.88%	0.048%
1/64	0.033%	2.31%	0.012%
1/128	0.0081%	1.15%	0.0029%
1/256	0.0020%	0.57%	0.0007%
1/512	0.0005%	0.28%	0.0002%

Figure 13 overlays the Maximum Error and Δx relationship diagrams of the above three methods and observes the differences. It can be found that when the mesh is very rough, the error of the Explicit Euler Method is very large compared to the other two methods. At the maximum error value, the error ratio is as high as 80.25%, which is at least 8 times worse than the 10.14% of

Gaussian Elimination and 3.36% of the Crank-Nicolson Method. Due to Explicit The Euler Method uses the slope of the current point to estimate the next point, so when the grid is very rough, the error will be particularly obvious. The other two methods will take multiple points into consideration, making it easier to respond to changes in the function, and the error will be relatively small. Among them, the Crank-Nicolson Method's 3.36% is the smallest. Observing the slope, we can see from the previous inversion that the errors of Gaussian Elimination and Crank-Nicolson Method both show quadratic convergence (O(Δx^2)), while Explicit Euler Method shows first-order convergence (O(Δx)). Therefore, in the process of cutting Δx into finer parts, the errors of Gaussian Elimination and Crank-Nicolson Method will decrease much faster than Explicit Euler Method. When the grid is cut to When Δx=1/512, the error ratio of Gaussian Elimination and Crank-Nicolson Method has dropped to the order of 10^(-4), and the Explicit Euler Method even stays at the order of 10^(-1). This conclusion can also be known by comparing Figure 14/Figure 15/Figure 16 and observing Table 7.

3.

(1) .

The element stiffness matrix of each element describes the relationship between the node displacement and the node force of this element, which can be expressed as , and since we are focusing on two-dimensional plane problems here, each node has degrees of freedom in two directions, and a four-node quadrilateral element has four nodes, so an element has a total of degrees of freedom, usually a matrix of . Finally, we merge the of each element into the stiffness matrix of the overall structure to get . The larger the number of nodes, the larger this matrix will be.$K^e K^e u^i = F^e 4 \times 2 = 8 K^e 8 \times 8 K^e KKU = FK$

After understanding the concept of matrix, we can observe its properties and summarize them as follows:K

- **Sparse matrix and Banded matrix:**

  After assembly, the matrix is a sparse matrix and a banded matrix, that is, most of the elements in the matrix are 0 and most of the non-0 items are

concentrated near the diagonal. The reason is that each element in the finite element method will only be connected to a few nodes near itself. For example, a four-node quadrilateral element will only affect the four nodes it is connected to, and the element's will only affect the degrees of freedom of these four points. In addition, when numbering, the node numbers are usually arranged according to their spatial positions, so when all are combined, non-0 Items will appear in the degree-of-freedom positions corresponding to nodes that are adjacent or belong to the same element, and since the node numbers are usually arranged according to their spatial positions, the numbers of these degrees of freedom are usually similar, so that non-zero items are concentrated near the main diagonal, and the area outside the vicinity of the diagonal will be 0. For example, suppose there is a 100×100 matrix that represents the rigid relationship between the 60th degree of freedom and the 40th degree of freedom. If the nodes corresponding to the two degrees of freedom are far apart in space and do not belong to the same element, there is usually no direct rigid relationship between them, so the value of this position is usually 0.$KK^eK^eKK_{60,40}$

- **Symmetric matrix:**

  After assembly, the matrix is a symmetric matrix, that is. Observing the formula, we can see that since the material matrix of general elastic materials is symmetric, the derived from this formula will also be symmetric. After merging into the matrix, it will be a symmetric matrix, which means that the displacement of the degree of freedom will affect the force of the degree of freedom, and in turn the displacement of the degree of freedom will also affect the force of the degree of freedom.$KK_{i,j} = K_{j,i}K^e = \int_{D_k} B^T DBdV\ DK^eK^eKijji$

- **Before there are no boundary conditions, the matrix is Positive semi-Definite (positive semi-definite), but after adding boundary conditions, the matrix becomes Positive Definite (positive definite):*KK***

  In structural mechanics, the strain energy is a vector composed of the

displacements of all nodes, and this strain energy will only be positive, and there will not be a situation where the energy after deformation is < 0 (the concept of producing energy without doing any work). However, when we add boundary conditions, it may represent the movement of the steel body (for example, the entire structure is displaced together). At this time, the overall structure will not deform, and the strain energy will be equal to 0, so the rigidity matrix has Positive semi-Definite properties and the system does not have a unique solution at this time. However, after we add boundary conditions (such as fixing the left end point of the cantilever beam), we can eliminate the possibility of steel body movement. The structure cannot be displaced and rotated together, so as long as is not a zero vector, it will be greater than 0, and the rigidity matrix will have the property of Positive Definite, and the system will have a unique solution at this time.$= \frac{1}{2} d^T K d d \rightarrow \frac{1}{2} d^T K d \geq 0 d K u_1 =$

$0, \quad u_2 = 0 d \frac{1}{2} d^T K d K$

(2) .

It can be known from (1) that the matrix is a Sparse, Banded, Symmetric matrix and is a Positive Definite matrix when boundary conditions are added. The reason why we do not use the general Gauss elimination to solve this linear system here is the waste of memory and calculation time. From Q2(b) above, we can see that for a matrix, the amount of operations using Gauss elimination is about . Therefore, in this way of forming a whole with many elements and then solving the overall linear system, the more nodes, the matrix will be too large, making the operation amount and operation time very large. In addition, the matrix is a Sparse matrix, and there are many 0 term, so when using Gauss elimination, you will need to use multiple partial pivotings to avoid division by 0, which affects the stability of the calculation. In terms of memory, most of the items in the Sparse matrix are 0. If we spend a lot of memory to store these 0s, the overall space usage will be very inefficient.$Kn \times nO(n^3)KK$

- **If the problem is not large, you can use sparse Cholesky**

decomposition:

If the boundary conditions are added, the matrix will become a Symmetric Positive Definite matrix. The direct method to solve this matrix is the Cholesky decomposition in the professor's briefing. This method first decomposes the matrix into , and the linear system will become . At this time, we assume that , first use Forward Substitution to calculate , and then use Backward Substitution to calculate . Here, since is a triangular matrix, both calculations are not complicated, and because the matrix is a Sparse matrix, if we use a simple Cholesky Decomposition will save all values (including 0 and non-0 items), which is very inefficient in space usage, so we use sparse Cholesky decomposition here to only access non-0 items to reduce memory waste.$KKK = LL^T \rightarrow LL^TU = FL^TU = yLy = FL^TU = yLK$

- **If the problem is very large, you can use the Conjugate Gradient method instead:**

  However, if the matrix is further expanded, the shortcomings of sparse Cholesky decomposition will become obvious. During calculation, although many positions of the matrix are 0, the corresponding position after decomposition is not necessarily 0, causing the matrix to be denser than the original matrix. At this time, when the scale of the problem becomes large (that is, the matrix becomes very large), the memory and calculation amount will become too large.$KLLKK$

  If the scale of the problem is large, the Conjugate Gradient method can be used instead. This method is suitable for use in large-scale sparse symmetric positive definite linear equations. The logic of its operation is to first guess a set of , then calculate the error and correct it in the appropriate direction. Repeat until the error is within the acceptable range, which is an iterative method. The reason why this method is suitable here is that each time the Conjugate Gradient method is calculated, it does not require a complete decomposition matrix. It only needs to repeatedly multiply the matrix vector. Since is a Sparse matrix, this kind of multiplication will be very fast.$Uresidue, r = F - KUUK$

(3) .

- **Displacement Error：**

  What we use here is a four-node quadrilateral element, and the displacement of each element can be weighted by its four nodes (that is, shape function,), and the shape function of these four nodes can be expressed as , if is larger, it can be seen that the node has a greater impact on the displacement of a certain element, and it can be found that these functions are all bilinear shape functions, that is, linear changes in the direction and in . This important property can assist us in predicting the error later. In addition, the position of node 1 in this natural coordinate is . If this coordinate is substituted, it can be found that at the position of node 1, and other are equal to 0. It can be seen that the displacement at the position of node 1 is completely controlled by node 1, and the same is true for nodes 2, 3, and 4. $N_i \begin{cases} N_1(\xi_1,\xi_2)=\frac{1}{4}(1-\xi_1)(1-\xi_2) \\ N_2(\xi_1,\xi_2)=\frac{1}{4}(1+\xi_1)(1-\xi_2) \\ N_3(\xi_1,\xi_2)=\frac{1}{4}(1+\xi_1)(1+\xi_2) \\ N_4(\xi_1,\xi_2)=\frac{1}{4}(1-\xi_1)(1+\xi_2) \end{cases}, \quad -1 \le \xi_1 \le 1, \quad -1 \le \xi_2 \le 1 N_i i \xi_1 \xi_2 (\xi_1,\xi_2) = (-1,-1) N_1 = 1 N_2, N_3, N_4$

  Assuming that the size of each element is , if we observe the Taylor expansion of the real displacement of a small segment of the element in the one-dimensional state, and from the above explanation, we can know that the approximate displacement of the element here can also be expressed as using the finite element method, that is, but because our shape function is bilinear, it is only a linear function in individual directions, so if we use numerical methods here, we will not be able to express the subsequent terms. This is the source of the error, and we can get it here. Displacement Error ie.$hu(x)u(x) \approx u(x_0) + u'(x_0)(x - x_0) + \frac{1}{2}u''(x_0)(x - x_0)^2 + H.O.T u_h(\xi_1,\xi_2) =$

  $\sum N_i(\xi_1,\xi_2)u_i$ numerical displacement $=$ shape function $\times$ nodal displacement $\frac{1}{2}u''(x_0)(x - x_0)^2 + H.O.T \|u - u_h\| \propto h^2 O(h^2)$

- **Strain/Stress Error：**

  It can be seen from the question that the strain is matrix multiplied by

node displacement, and inside the matrix is the derivative of the shape function to the space, which can be expressed as , and from the above derivation, it can be seen that the displacement error is . After we differentiate this on the space once, we can get , and the error order will be reduced by one order. It can be seen that strain error = .$BB \rightarrow$

$$\begin{bmatrix} \epsilon_1(x_1,x_2) \\ \epsilon_2(x_1,x_2) \\ \gamma_{12}(x_1,x_2) \end{bmatrix} =$$

$$\begin{bmatrix} \frac{\partial N_1}{\partial x_1} & 0 & \frac{\partial N_2}{\partial x_1} & 0 & \frac{\partial N_3}{\partial x_1} & 0 & \frac{\partial N_4}{\partial x_1} & 0 \\ 0 & \frac{\partial N_1}{\partial x_2} & 0 & \frac{\partial N_2}{\partial x_2} & 0 & \frac{\partial N_3}{\partial x_2} & 0 & \frac{\partial N_4}{\partial x_2} \\ \frac{\partial N_1}{\partial x_2} & \frac{\partial N_1}{\partial x_1} & \frac{\partial N_2}{\partial x_2} & \frac{\partial N_2}{\partial x_1} & \frac{\partial N_3}{\partial x_2} & \frac{\partial N_3}{\partial x_1} & \frac{\partial N_4}{\partial x_2} & \frac{\partial N_4}{\partial x_1} \end{bmatrix} \begin{bmatrix} u_1^1 \\ u_2^1 \\ u_1^2 \\ u_2^2 \\ u_1^3 \\ u_2^3 \\ u_1^4 \\ u_2^4 \end{bmatrix} O(h^2) \rightarrow \frac{\partial(u-u_h)}{\partial x} \propto$$

$hO(h)$

The stress part can be seen from the question. Here is the material matrix, which does not converge to the order, so the stress error will be of the same order as the strain error, that is, stress error = .$\sigma = D\epsilon DO(h)$

(4) .

Here we use MATLAB to assist in problem simulation, error calculation and analysis. The grid adopts . If it is finer, the computer will be unable to run. Referring to the system side length ratio set by the question, this grid number ratio will make each element a square with a side length of .$(N_x, N_y) =$ $(6,2)$ 、$(12,4)$ 、$(24,8)$ 、$(48,16)$ 、$(96,32) N_x: N_y = 3:1 h$

Here we output the number of grids, element side lengths, Numerical Tip Displacement (numeric solution displacement), and Exact Tip Displacement (positive solution displacement). In the error part, in addition to outputting the errors of the FEM solution and the positive solution, we also output the errors of each coarse grid FEM solution and the finest grid FEM solution using the finest grid () as a reference, and observe and analyze the performance of these two errors.$h 96 \times 32$

Table 8. Displacement and error values under different grids

N_x	N_y	$h\ (mm)$	Numerical solution maximum displacement (mm)	Correct solution maximum displacement (mm)	Maximum error of correct solution (mm)	Finest mesh reference error (mm)
6	2	10	8.1981	9.3888	1.1907	1.1106
12	4	5	8.9855	9.3888	0.4033	0.3232
24	8	2.5	9.2237	9.3888	0.1651	0.08506
48	16	1.25	9.2901	9.3888	0.09865	0.01858
96	32	0.625	9.3087	9.3888	0.08008	NaN

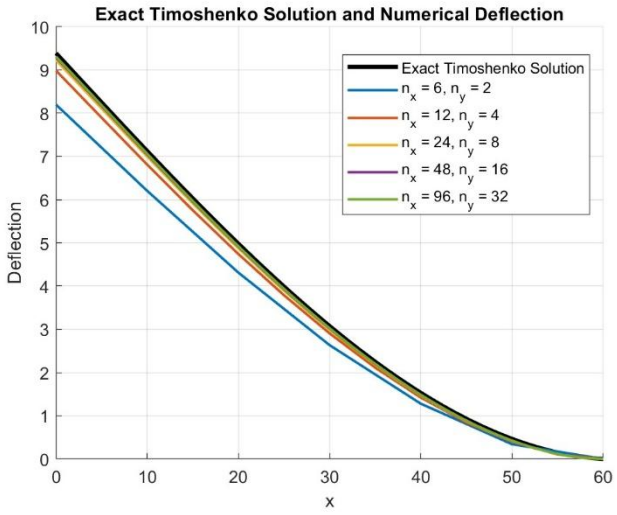


Figure 17. Displacement and position relationship diagram under different grids

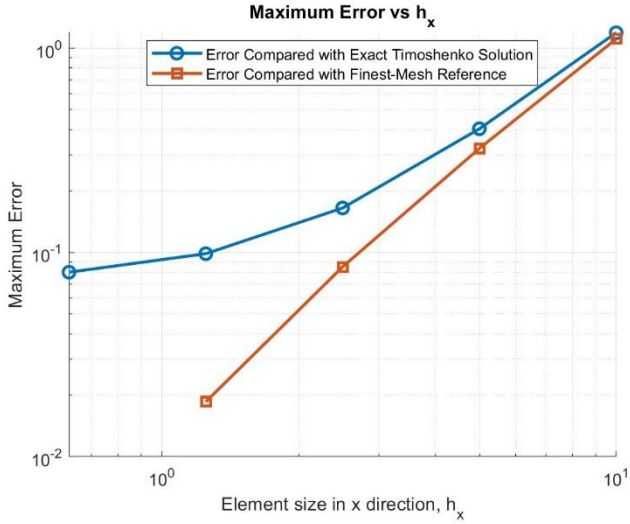


Figure 18. Relationship between maximum error and element side length$h$

Table 8 calculates the error values under different grids. It can be found that

when the grid is finer, the error between the numerical solution and the correct solution will be smaller. And it can be seen from Figure 17 that when the grid density increases, the relationship between the displacement and the position will be closer to the curve of the correct solution. When the grid is the roughest (i.e.), the maximum error ratio of the correct solution is as high as , and when the grid is the densest (i.e.), the maximum error ratio of the correct solution is only At this time, the curve in Figure 17 has roughly coincided with the correct solution curve.$6 \times 2\frac{1.1907}{9.3888} = 12.682\%96 \times 32\frac{0.08008}{9.3888} = 0.853\%96 \times 32$

In Figure 18, we output the errors of the FEM solution and the positive solution and the errors of each coarse-grid FEM solution and the finest-grid FEM solution respectively and stack the graphs for comparison. It can be found that in the log-log plot, the errors of the FEM solution and the positive solution do not appear in a straight line. From Table 8, we can also find that every time when drops to 1/2 of the original, the error every time will drop to the original. 0.34, 0.41, 0.60, 0.81, the decrease amplitude shows a flat characteristic. When is smaller, the error decrease amplitude will become smaller, and there is no Displacement Error = relationship we deduced earlier. However, in the error relationship between each coarse-grid FEM solution and the finest-grid FEM solution, we can find that it presents a straight line with a slope of approximately 2 in the log-log plot. From Table 8, it can be seen that each time when drops to 1/2 of the original, the error each time drops to approximately 0.29, 0.26, and 0.22 of the original. Although the decrease ratio is slightly different from 0.25, we can roughly see the relationship of Displacement Error =.$hhO(h^2)hO(h^2)$

It is speculated that the reason for the above phenomenon is that when analyzing the error between the FEM solution and the forward solution, the Timoshenko beam solution method I used to calculate the forward solution here is more suitable for calculating the one-dimensional beam model. There are some model differences from the two-dimensional plane problem solved by FEM. As a result, when calculating the maximum error of the forward solution, it will include discrete errors and model difference errors. According

to theory, when the element side length When it drops to a very small value, the discrete error will quickly converge at a quadratic speed, but the model difference error will not disappear as the grid becomes thinner, so the overall maximum error of the correct solution will gradually level off. But when we use the finest grid as a reference to calculate the error of each coarse grid FEM solution and the finest grid FEM solution, since the two calculations use the same model, the model difference error mentioned above will disappear, so that the reference error of the finest grid in the table here is only a discrete error, and we can see the Displacement Error = relationship derived above.$hO(h^2)$

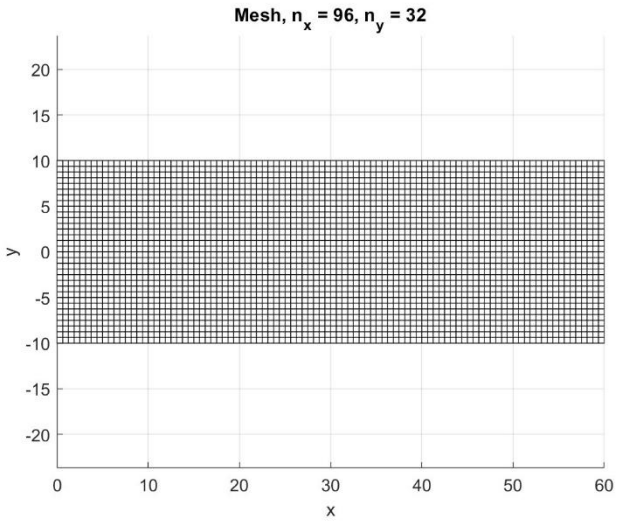


Figure 19. xy position map of the densest grid

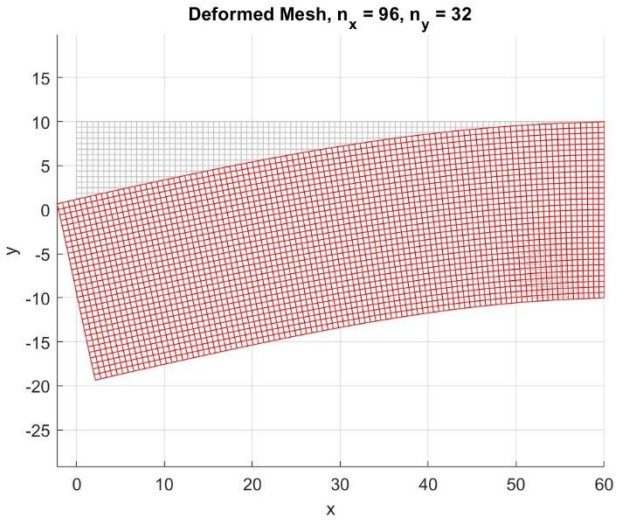


Figure 20. The xy position diagram of the densest grid when it is stressed

Here we also express the stress and non-stress conditions when the grid is densest with the most intuitive xy position diagram. If we observe the position of , we can find that the vertical displacement of the center point is consistent with the value we obtained earlier, and close to , the displacement of the element is very small due to the defined boundary conditions. These two pictures allow us to very intuitively observe the displacement of its elements when the tail end of a cantilever is stressed.$x = 0 x = 1$

Table 9. Strain RMS error of the finest mesh reference under different meshes

N_x	N_y	$h\ (mm)$	Strain RMS error of the finest mesh reference
6	2	10	2.391×10^{-3}
12	4	5	8.738×10^{-4}
24	8	2.5	2.799×10^{-4}
48	16	1.25	1.621×10^{-4}
96	32	0.625	NaN

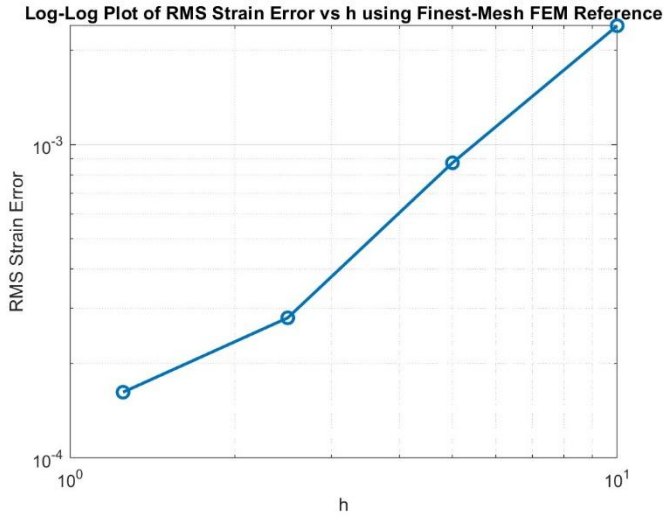


Figure 21. The relationship between the strain RMS error of the finest mesh reference and$h$

We output the relationship between strain and element side length. We also refer to the previous method and use the value obtained by the finest mesh as a reference to calculate the error. This way we can focus on the error phenomenon after pure discretization and will not be affected by errors caused by other reasons. The reason why the maximum value is not used to calculate

the error here as before is because the strain error is easily amplified by the influence of local positions. However, if RMS is used to calculate the error, the overall error can be averaged and the chance of local points controlling the overall error can be reduced.h

It can be seen here that in the log-log diagram, the strain RMS error of the finest grid reference and the element side length roughly form a straight line with a slope of 1, which is consistent with the previous theoretical derivation, that is, strain error = . However, there are still some outliers that affect the overall error value. In addition, the calculated strain will differentiate the displacement, resulting in an amplification of the error. These unstable reasons make the strain error not displayed as a perfect straight line in the diagram, and there will be slight deviations in some places.$hO(h)$

Table 10. Stress RMS error of the finest mesh reference under different meshes

N_x	N_y	$h\ (mm)$	Stress RMS error of the finest mesh reference (N/mm2)
6	2	10	2.193×10^2
12	4	5	7.334×10^1
24	8	2.5	2.194×10^1
48	16	1.25	1.814×10^1
96	32	0.625	NaN

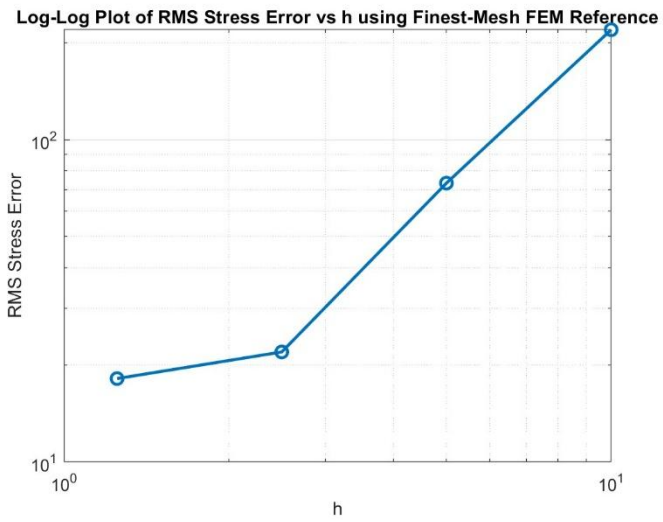


Figure 22. The relationship between the stress RMS error of the finest mesh reference and$h$

Here, the relationship between the stress RMS error of the finest grid reference and the element side length is also output. It can be found that it has roughly the same trend as the strain, which is roughly consistent with the theoretical stress error =.$hO(h)$

Appendices

AI is used here to assist in the writing of the program, and the AI is strictly required to "only output the program code to me without any text description."

AI conversation link: https://chatgpt.com/share/6a26fb07-1990-83a6-acb3-9d6f1a86e24e

B12502027_Q2_b_(ii)

Prompt (Attach all instructions in order):

1. $$\begin{bmatrix} b & c & 0 & & 0 & 0 & 0 \\ a & b & c & \cdots & 0 & 0 & 0 \\ 0 & a & b & & 0 & 0 & 0 \\ & \vdots & & \ddots & & \vdots & \\ 0 & 0 & 0 & & b & c & 0 \\ 0 & 0 & 0 & \cdots & a & b & c \\ 0 & 0 & 0 & & 0 & a & b \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-4} \\ y_{N-3} \\ y_{N-2} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{N-4} \\ x_{N-3} \\ x_{N-2} - 2c \end{bmatrix}$$

   a=(1/(Δx^2)-2/Δx),b=(4-2/(Δx^2)),c=(1/(Δx^2)+2/Δx)

   The above is the set of linear equations I obtained. Please help me write a program code using MATLAB to solve this set of linear equations using Gaussian elimination method (Don't use the backslash function (\) in Matlab to solve the set of linear equations). If my Δx is a variable, let me observe the performance of the error under different Δx conditions. Just output the program code to me without any text description.
2. Help me express the obtained solution and the value of the analytical solution, not just the error
3. Also output the residual value after each group of different dx calculations
4. Output a graph with dx=1/512. The horizontal axis is x and the vertical axis is error.
5. Output this picture of all different dx
6. Help me output a stacked graph that contains analytical solutions and solutions calculated under different dx.
7. Just draw a line without marking every point.
8. When outputting this picture with all different dx, please help me adjust the vertical axis so that the butterfly pictures will not all stick together.
9. Output a graph with dx=1/64. The horizontal axis is x and the vertical axis is error.
10. show the points

(After entering all the above prompts, AI will only generate the program code for me)

The final code (some pictures in the report are generated by using the code generated by several prompts in the middle, and are not output by the final code):

```
clear; clc; close all;

dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];

error_list = zeros(size(dx_list));
residual_list = zeros(size(dx_list));

figure;
hold on;
grid on;

x_exact_plot = linspace(0, 1, 1000);
y_exact_plot = exactSolution(x_exact_plot);

plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...
    'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

    dx = dx_list(k);

    N = round(1/dx) + 1;
    n = N - 2;

    x = (1:n)' * dx;

    a = 1/(dx^2) - 2/dx;
    b = 4 - 2/(dx^2);
    c = 1/(dx^2) + 2/dx;

    A = zeros(n, n);

    for i = 1:n
        A(i,i) = b;

        if i > 1
            A(i,i-1) = a;
        end
```

```
        if i < n
            A(i,i+1) = c;
        end
    end

    rhs = x;
    rhs(end) = rhs(end) - 2*c;

    y_numerical = gaussianElimination(A, rhs);

    y_exact = exactSolution(x);
    abs_error = abs(y_numerical - y_exact);

    residual = norm(A*y_numerical - rhs, inf);
    error = norm(abs_error, inf);

    residual_list(k) = residual;
    error_list(k) = error;

    fprintf('\n========================================\n');
    fprintf('dx = %.8f\n', dx);
    fprintf('Residual = %.12e\n', residual);
    fprintf('Maximum Error = %.12e\n', error);
    fprintf('========================================\n');

    result_table = table(x, y_numerical, y_exact, abs_error, ...
        'VariableNames', {'x', 'Numerical_y', 'Exact_y', 'Absolute_Error'});

    disp(result_table);

    x_with_bc = [0; x; 1];
    y_with_bc = [0; y_numerical; 2];

    plot(x_with_bc, y_with_bc, '-o', ...
        'DisplayName', ['Numerical, \Deltax = ', num2str(dx)]);

end
```

```
xlabel('x');
ylabel('y');
title('Exact Solution and Numerical Solutions for Different \Deltax');
legend('show', 'Location', 'best');

summary_table = table(dx_list', error_list', residual_list', ...
    'VariableNames', {'dx', 'Max_Error', 'Residual'});

fprintf('\n=========================================\n');
fprintf('Summary Table\n');
fprintf('=========================================\n');

disp(summary_table);

figure;
hold on;
grid on;

for k = 1:length(dx_list)

    dx = dx_list(k);

    N = round(1/dx) + 1;
    n = N - 2;

    x = (1:n)' * dx;

    a = 1/(dx^2) - 2/dx;
    b = 4 - 2/(dx^2);
    c = 1/(dx^2) + 2/dx;

    A = zeros(n, n);

    for i = 1:n
        A(i,i) = b;

        if i > 1
```

```
            A(i,i-1) = a;
        end

        if i < n
            A(i,i+1) = c;
        end
    end

    rhs = x;
    rhs(end) = rhs(end) - 2*c;

    y_numerical = gaussianElimination(A, rhs);
    y_exact = exactSolution(x);

    abs_error = abs(y_numerical - y_exact);

    plot(x, abs_error, '-o', ...
        'DisplayName', ['\Deltax = ', num2str(dx)]);

end

xlabel('x');
ylabel('Error');
title('Error vs x for Different \Deltax');
legend('show', 'Location', 'best');

figure;
loglog(dx_list, error_list, '-o');
grid on;
xlabel('\Deltax');
ylabel('Maximum Error');
title('Maximum Error vs \Deltax');

figure;
loglog(dx_list, residual_list, '-o');
grid on;
xlabel('\Deltax');
ylabel('Residual');
```

```
title('Residual vs \Deltax');


function y = gaussianElimination(A, b)

    n = length(b);
    Aug = [A b];

    for k = 1:n-1

        [~, pivotRow] = max(abs(Aug(k:n,k)));
        pivotRow = pivotRow + k - 1;

        if pivotRow ~= k
            temp = Aug(k,:);
            Aug(k,:) = Aug(pivotRow,:);
            Aug(pivotRow,:) = temp;
        end

        for i = k+1:n
            factor = Aug(i,k) / Aug(k,k);
            Aug(i,k:end) = Aug(i,k:end) - factor * Aug(k,k:end);
        end

    end

    y = zeros(n,1);

    for i = n:-1:1
        y(i) = (Aug(i,end) - Aug(i,i+1:n) * y(i+1:n)) / Aug(i,i);
    end

end


function y = exactSolution(x)

    C1 = 1/4;
```

```
    C2 = 2*exp(2) - 1/4;

    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

end
```

B12502027_Q2_c_(iii)

Prompt (Attach all instructions in order):

1. $y_n = u_{1,n} + \frac{2-u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \cdots, N \quad \begin{cases} u_{1,n+1} = u_{1,n} + \Delta x u_{2,n} \\ u_{2,n+1} = u_{2,n} + \Delta x(-4u_{2,n} - 4u_{1,n} + x) \end{cases} \quad \begin{cases} v_{1,n+1} = v_{1,n} + \Delta x v_{2,n} \\ v_{2,n+1} = v_{2,n} + \Delta x(-4v_{2,n} - 4v_{1,n}) \end{cases}$

   The above is the solution I obtained after substituting Explicit Euler Method into two IVPs. Help me write a program code calculation using MATLAB. Here u_1 is a non-homogeneous solution and v_1 is a homogeneous solution. If my Δx is a variable, let me observe the performance under different Δx situations. Just output the program code to me without any text description.
2. There is no need to output the residual value. The error vs.
3. What I mean is that the graph of error versus x helps me output each dx separately, but the graph of y versus x is still the same stacked graph.
4. Output a graph with dx=1/512. The horizontal axis is x and the vertical axis is error.

(After entering all the above prompts, AI will only generate the program code for me)

The final code (some pictures in the report are generated by using the code generated by several prompts in the middle, and are not output by the final code):

```
clear; clc; close all;

dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];

error_list = zeros(size(dx_list));
residual_list = zeros(size(dx_list));

figure;
hold on;
grid on;

x_exact_plot = linspace(0, 1, 1000)';
y_exact_plot = exactSolution(x_exact_plot);
```

```
plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...
    'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

    dx = dx_list(k);
    N = round(1/dx);

    x = (0:N)' * dx;

    u1 = zeros(N+1, 1);
    u2 = zeros(N+1, 1);
    v1 = zeros(N+1, 1);
    v2 = zeros(N+1, 1);

    u1(1) = 0;
    u2(1) = 0;

    v1(1) = 0;
    v2(1) = 1;

    for n = 1:N
        u1(n+1) = u1(n) + dx*u2(n);
        u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

        v1(n+1) = v1(n) + dx*v2(n);
        v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));
    end

    theta = (2 - u1(end)) / v1(end);

    y_numerical = u1 + theta*v1;
    y_exact = exactSolution(x);

    abs_error = abs(y_numerical - y_exact);
    max_error = norm(abs_error, inf);
```

```
        residual = abs(y_numerical(end) - 2);

        error_list(k) = max_error;
        residual_list(k) = residual;

        fprintf('\n=========================================\n');
        fprintf('dx = %.8f\n', dx);
        fprintf('theta = %.12e\n', theta);
        fprintf('Residual = %.12e\n', residual);
        fprintf('Maximum Error = %.12e\n', max_error);
        fprintf('=========================================\n');

        result_table = table(x, u1, v1, y_numerical, y_exact, abs_error, ...
            'VariableNames', {'x', 'u1_nonhomogeneous', 'v1_homogeneous', ...
            'Numerical_y', 'Exact_y', 'Absolute_Error'});

        disp(result_table);

        plot(x, y_numerical, '-', 'LineWidth', 1.5, ...
            'DisplayName', ['Numerical, \Deltax = ', num2str(dx)]);

end

xlabel('x');
ylabel('y');
title('Exact Solution and Explicit Euler Shooting Solutions');
legend('show', 'Location', 'best');

summary_table = table(dx_list', error_list', residual_list', ...
    'VariableNames', {'dx', 'Max_Error', 'Residual'});

fprintf('\n=========================================\n');
fprintf('Summary Table\n');
fprintf('=========================================\n');

disp(summary_table);

figure;
```

```
hold on;
grid on;

for k = 1:length(dx_list)

    dx = dx_list(k);
    N = round(1/dx);

    x = (0:N)' * dx;

    u1 = zeros(N+1, 1);
    u2 = zeros(N+1, 1);
    v1 = zeros(N+1, 1);
    v2 = zeros(N+1, 1);

    u1(1) = 0;
    u2(1) = 0;

    v1(1) = 0;
    v2(1) = 1;

    for n = 1:N
        u1(n+1) = u1(n) + dx*u2(n);
        u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

        v1(n+1) = v1(n) + dx*v2(n);
        v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));
    end

    theta = (2 - u1(end)) / v1(end);

    y_numerical = u1 + theta*v1;
    y_exact = exactSolution(x);

    abs_error = abs(y_numerical - y_exact);

    plot(x, abs_error, '-', 'LineWidth', 1.5, ...
        'DisplayName', ['\Deltax = ', num2str(dx)]);
```

```
end

xlabel('x');
ylabel('Error');
title('Error vs x for Different \Deltax');
legend('show', 'Location', 'best');

figure;
loglog(dx_list, error_list, '-o', 'LineWidth', 1.5);
grid on;
xlabel('\Deltax');
ylabel('Maximum Error');
title('Maximum Error vs \Deltax');

figure;
loglog(dx_list, residual_list, '-o', 'LineWidth', 1.5);
grid on;
xlabel('\Deltax');
ylabel('Residual');
title('Residual vs \Deltax');



function y = exactSolution(x)

    C1 = 1/4;
    C2 = 2*exp(2) - 1/4;

    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

end
```

B12502027_Q2_c_(iv)

Prompt (Attach all instructions in order):

1. 令 $M_1 = I - \frac{\Delta x}{2}A, \quad M_2 = I + \frac{\Delta x}{2}A$

$$\rightarrow \begin{cases} M_1 u_{n+1} = M_2 u_n + \frac{\Delta x}{2}(B(x_n) + B(x_{n+1})) \\ M_1 v_{n+1} = M_2 v_n \end{cases} \qquad y_n = u_{1,n} + \frac{2 - u_{1,N}}{v_{1,N}} v_{1,n}, \quad n = 0, 1, 2, \cdots, N$$

The above is the solution I obtained after substituting the Crank-Nicolson Method into two IVPs. Help me write a program code calculation using MATLAB. If my Δx is a variable, let me observe the error performance under different Δx conditions. Just output the program code to me without any text description.

(After entering all the above prompts, AI will only generate the program code for me)

The final code (some pictures in the report are generated by using the code generated by several prompts in the middle, and are not output by the final code):

```
clear; clc; close all;

dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];

error_list = zeros(size(dx_list));

A = [0 1;
     -4 -4];

fig_y = figure;
hold on;
grid on;

x_exact_plot = linspace(0, 1, 1000)';
y_exact_plot = exactSolution(x_exact_plot);

plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 2.5, ...
     'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)

     dx = dx_list(k);
     N = round(1/dx);

     x = (0:N)' * dx;
```

```
u = zeros(2, N+1);
v = zeros(2, N+1);

u(:,1) = [0; 0];
v(:,1) = [0; 1];

M1 = eye(2) - dx/2 * A;
M2 = eye(2) + dx/2 * A;

for n = 1:N

    Bn   = [0; x(n)];
    Bnp1 = [0; x(n+1)];

    rhs_u = M2*u(:,n) + dx/2 * (Bn + Bnp1);
    rhs_v = M2*v(:,n);

    u(:,n+1) = solve2by2(M1, rhs_u);
    v(:,n+1) = solve2by2(M1, rhs_v);

end

u1 = u(1,:)';
v1 = v(1,:)';

theta = (2 - u1(end)) / v1(end);

y_numerical = u1 + theta*v1;
y_exact = exactSolution(x);

abs_error = abs(y_numerical - y_exact);
max_error = norm(abs_error, inf);

error_list(k) = max_error;

fprintf('\n=========================================\n');
fprintf('dx = %.8f\n', dx);
```

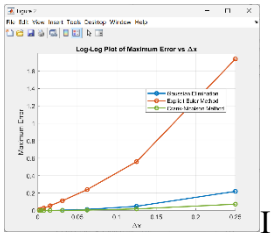
```
        fprintf('theta = %.12e\n', theta);
        fprintf('Maximum Error = %.12e\n', max_error);
        fprintf('========================================\n');

        result_table = table(x, u1, v1, y_numerical, y_exact, abs_error, ...
            'VariableNames', {'x', 'u1_nonhomogeneous', 'v1_homogeneous', ...
            'Numerical_y', 'Exact_y', 'Absolute_Error'});

        disp(result_table);

        figure(fig_y);
        plot(x, y_numerical, '-', 'LineWidth', 1.5, ...
            'DisplayName', ['Numerical, \Deltax = ', num2str(dx)]);

        figure;
        plot(x, abs_error, '-o', 'LineWidth', 1.5, 'MarkerSize', 5);
        grid on;
        xlabel('x');
        ylabel('Error');
        title(['Error vs x when \Deltax = ', num2str(dx)]);

end

figure(fig_y);
xlabel('x');
ylabel('y');
title('Exact Solution and Crank-Nicolson Shooting Solutions');
legend('show', 'Location', 'best');

summary_table = table(dx_list', error_list', ...
    'VariableNames', {'dx', 'Maximum_Error'});

fprintf('\n========================================\n');
fprintf('Summary Table\n');
fprintf('========================================\n');

disp(summary_table);
```

```
figure;
loglog(dx_list, error_list, '-o', 'LineWidth', 1.5, 'MarkerSize', 6);
grid on;
xlabel('\Deltax');
ylabel('Maximum Error');
title('Maximum Error vs \Deltax');


function x = solve2by2(A, b)

    detA = A(1,1)*A(2,2) - A(1,2)*A(2,1);

    x = zeros(2,1);
    x(1) = ( b(1)*A(2,2) - A(1,2)*b(2)) / detA;
    x(2) = ( A(1,1)*b(2) - b(1)*A(2,1)) / detA;

end


function y = exactSolution(x)

    C1 = 1/4;
    C2 = 2*exp(2) - 1/4;

    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

end
```

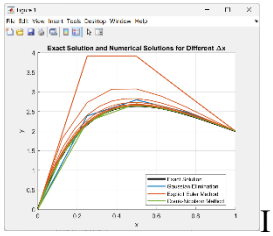
B12502027_Q2_dicussion

Prompt (Attach all instructions in order):

1. Write a MATLAB program code to overlay the Maximum Error and Δx relationship diagrams of the three methods Gaussian Elimination, Explicit Euler Method, and Crank-Nicolson Method, the analytical solutions, and the comparison diagrams of numerical solutions under different Δx. As long as the program code
2. The graph of the relationship between Maximum Error and Δx produces a log-log graph, the analytical solution and the comparison graph of the numerical solution under different Δx. The same method is represented in the same color, and then the three graphs are stacked. I want to see the convergence trend.

3. 

I want this log-log picture

4. 

I want to use the same method and the same color for this picture.

(After entering all the above prompts, AI will only generate the program code for me)

The final code (some pictures in the report are generated by using the code generated by several prompts in the middle, and are not output by the final code):

```
clear; clc; close all;

dx_list = [1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512];

err_GE = zeros(size(dx_list));
err_EE = zeros(size(dx_list));
err_CN = zeros(size(dx_list));

color_GE = [0.0000 0.4470 0.7410];
color_EE = [0.8500 0.3250 0.0980];
color_CN = [0.4660 0.6740 0.1880];

x_exact_plot = linspace(0,1,2000)';
y_exact_plot = exactSolution(x_exact_plot);

figure(1);
hold on; grid on;
plot(x_exact_plot, y_exact_plot, 'k-', 'LineWidth', 3, ...
     'DisplayName', 'Exact Solution');

for k = 1:length(dx_list)
```

```
    dx = dx_list(k);

    [x_GE, y_GE] = GaussianEliminationMethod(dx);
    [x_EE, y_EE] = ExplicitEulerMethod(dx);
    [x_CN, y_CN] = CrankNicolsonMethod(dx);

    err_GE(k) = norm(y_GE - exactSolution(x_GE), inf);
    err_EE(k) = norm(y_EE - exactSolution(x_EE), inf);
    err_CN(k) = norm(y_CN - exactSolution(x_CN), inf);

    if k == 1
        plot(x_GE, y_GE, '-', 'Color', color_GE, 'LineWidth', 1.5, ...
            'DisplayName', 'Gaussian Elimination');
        plot(x_EE, y_EE, '-', 'Color', color_EE, 'LineWidth', 1.5, ...
            'DisplayName', 'Explicit Euler Method');
        plot(x_CN, y_CN, '-', 'Color', color_CN, 'LineWidth', 1.5, ...
            'DisplayName', 'Crank-Nicolson Method');
    else
        plot(x_GE, y_GE, '-', 'Color', color_GE, 'LineWidth', 1.5, ...
            'HandleVisibility', 'off');
        plot(x_EE, y_EE, '-', 'Color', color_EE, 'LineWidth', 1.5, ...
            'HandleVisibility', 'off');
        plot(x_CN, y_CN, '-', 'Color', color_CN, 'LineWidth', 1.5, ...
            'HandleVisibility', 'off');
    end

end

xlabel('x');
ylabel('y');
title('Exact Solution and Numerical Solutions for Different \Deltax');
legend('show', 'Location', 'best');


figure(2);
hold on; grid on;

loglog(dx_list, err_GE, '-o', 'Color', color_GE, ...
```

```
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Gaussian Elimination');

loglog(dx_list, err_EE, '-o', 'Color', color_EE, ...
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Explicit Euler Method');

loglog(dx_list, err_CN, '-o', 'Color', color_CN, ...
        'LineWidth', 1.8, 'MarkerSize', 6, ...
        'DisplayName', 'Crank-Nicolson Method');

set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');

xlabel('\Deltax');
ylabel('Maximum Error');
title('Log-Log Plot of Maximum Error vs \Deltax');
legend('show', 'Location', 'best');
xticks(dx_list);
xticklabels({'1/4','1/8','1/16','1/32','1/64','1/128','1/256','1/512'});

summary_table = table(dx_list', err_GE', err_EE', err_CN', ...
        'VariableNames', {'dx', 'Gaussian_Elimination_Error', ...
        'Explicit_Euler_Error', 'Crank_Nicolson_Error'});

disp(summary_table);


function [x_all, y_all] = GaussianEliminationMethod(dx)

        N = round(1/dx) + 1;
        n = N - 2;

        x = (1:n)' * dx;

        a = 1/(dx^2) - 2/dx;
        b = 4 - 2/(dx^2);
        c = 1/(dx^2) + 2/dx;
```

```
    A = zeros(n, n);

    for i = 1:n
        A(i,i) = b;
        if i > 1
            A(i,i-1) = a;
        end
        if i < n
            A(i,i+1) = c;
        end
    end

    rhs = x;
    rhs(end) = rhs(end) - 2*c;

    y = gaussianElimination(A, rhs);

    x_all = [0; x; 1];
    y_all = [0; y; 2];

end


function [x, y] = ExplicitEulerMethod(dx)

    N = round(1/dx);
    x = (0:N)' * dx;

    u1 = zeros(N+1,1);
    u2 = zeros(N+1,1);
    v1 = zeros(N+1,1);
    v2 = zeros(N+1,1);

    u1(1) = 0;
    u2(1) = 0;
    v1(1) = 0;
    v2(1) = 1;
```

```
    for n = 1:N
        u1(n+1) = u1(n) + dx*u2(n);
        u2(n+1) = u2(n) + dx*(-4*u2(n) - 4*u1(n) + x(n));

        v1(n+1) = v1(n) + dx*v2(n);
        v2(n+1) = v2(n) + dx*(-4*v2(n) - 4*v1(n));
    end

    theta = (2 - u1(end)) / v1(end);
    y = u1 + theta*v1;

end


function [x, y] = CrankNicolsonMethod(dx)

    N = round(1/dx);
    x = (0:N)' * dx;

    A = [0 1; -4 -4];
    M1 = eye(2) - dx/2*A;
    M2 = eye(2) + dx/2*A;

    u = zeros(2, N+1);
    v = zeros(2, N+1);

    u(:,1) = [0; 0];
    v(:,1) = [0; 1];

    for n = 1:N
        Bn   = [0; x(n)];
        Bnp1 = [0; x(n+1)];

        rhs_u = M2*u(:,n) + dx/2*(Bn + Bnp1);
        rhs_v = M2*v(:,n);

        u(:,n+1) = solve2by2(M1, rhs_u);
```

```
        v(:,n+1) = solve2by2(M1, rhs_v);
    end

    u1 = u(1,:)';
    v1 = v(1,:)';

    theta = (2 - u1(end)) / v1(end);
    y = u1 + theta*v1;

end


function y = gaussianElimination(A, b)

    n = length(b);
    Aug = [A b];

    for k = 1:n-1
        [~, pivotRow] = max(abs(Aug(k:n,k)));
        pivotRow = pivotRow + k - 1;

        if pivotRow ~= k
            temp = Aug(k,:);
            Aug(k,:) = Aug(pivotRow,:);
            Aug(pivotRow,:) = temp;
        end

        for i = k+1:n
            factor = Aug(i,k) / Aug(k,k);
            Aug(i,k:end) = Aug(i,k:end) - factor * Aug(k,k:end);
        end
    end

    y = zeros(n,1);

    for i = n:-1:1
        y(i) = (Aug(i,end) - Aug(i,i+1:n)*y(i+1:n)) / Aug(i,i);
    end
```

```
end


function x = solve2by2(A, b)

    detA = A(1,1)*A(2,2) - A(1,2)*A(2,1);

    x = zeros(2,1);
    x(1) = (b(1)*A(2,2) - A(1,2)*b(2)) / detA;
    x(2) = (A(1,1)*b(2) - b(1)*A(2,1)) / detA;

end


function y = exactSolution(x)

    C1 = 1/4;
    C2 = 2*exp(2) - 1/4;
    y = (C1 + C2*x).*exp(-2*x) + (x - 1)/4;

end
```

B12502027_Q3_Q4

Prompt (Attach all instructions in order):

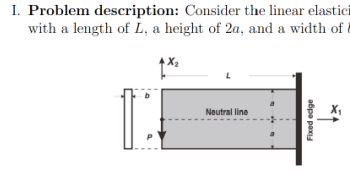

I. **Problem description:** Consider the linear elasticity of a cantilever beam with a length of L, a height of $2a$, and a width of b as shown in figure 1.

Figure 1: Schematics of the cantilevel beam.

1. The above is our topic. Write a MATLAB code to help me simulate using the finite element method. If my element size h is an adjustable variable, let me observe the error performance at different h. As long as the code does not explain it.

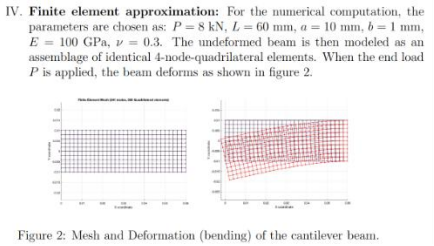
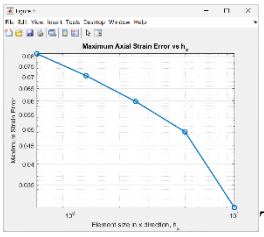

IV. **Finite element approximation:** For the numerical computation, the parameters are chosen as: $P = 8$ kN, $L = 60$ mm, $a = 10$ mm, $b = 1$ mm, $E = 100$ GPa, $\nu = 0.3$. The undeformed beam is then modeled as an assemblage of identical 4-node-quadrilateral elements. When the end load P is applied, the beam deforms as shown in figure 2.

Figure 2: Mesh and Deformation (bending) of the cantilever beam.

2. The element size h means that I can set the number of grids in the x and y directions.
3. I also need to output the exact solution so that I can observe the error between the numerical solution and the correct solution.
4. Graph Exact Solution and Numerical Deflection with different number of grids
5. The exact solution is wrong, change the exact solution to Timoshenko beam form and re-enter the code.
6. Take the finest-mesh reference error into consideration
7. nx_list = [6, 12, 24, 48, 96]; ny_list = [2, 4, 8, 16, 32]; Also output the mesh map
8. There is no need to output the mesh diagram now, but help me output the analytical solution of strain and the numerical solution error.
9. 

This log-log picture
10. Change to max strain error under different h
11. I mean changing the overall code to the max strain error under different h
12. Make a log-log plot of the strain error of different h versus h.
13. Make a log-log plot of the stress error of different h versus h.
14. Make the strain error of different h versus the log-log plot of h and use the finest-mesh FEM strain as a reference to calculate the error.
15. Use RMS error instead
16. Also display the stress

(After entering all the above prompts, AI will only generate the program code for me)

The final code (some pictures in the report are generated by using the code generated by several prompts in the middle, and are not output by the final code):

```
clear; clc; close all;


P  = 8000;
L  = 60;
a  = 10;
b  = 1;
E  = 100000;
nu = 0.3;
```

```
nx_list = [6, 12, 24, 48, 96];
ny_list = [2, 4, 8, 16, 32];

rms_strain_error_ref_list = zeros(size(nx_list));
rms_stress_error_ref_list = zeros(size(nx_list));

hx_list = zeros(size(nx_list));
hy_list = zeros(size(nx_list));
h_list    = zeros(size(nx_list));

D = E/(1-nu^2) * [1 nu 0;
                  nu 1 0;
                  0    0 (1-nu)/2];

all_x_strain = cell(length(nx_list),1);
all_y_strain = cell(length(nx_list),1);
all_eps_num    = cell(length(nx_list),1);
all_sig_num    = cell(length(nx_list),1);

for m = 1:length(nx_list)

    nx = nx_list(m);
    ny = ny_list(m);

    hx = L / nx;
    hy = 2*a / ny;
    h    = max(hx, hy);

    hx_list(m) = hx;
    hy_list(m) = hy;
    h_list(m)    = h;

    [node, elem] = generateMesh(L, a, nx, ny);

    numNode = size(node,1);
    numElem = size(elem,1);
    numDOF    = 2*numNode;
```

```
K = zeros(numDOF, numDOF);
F = zeros(numDOF, 1);

for e = 1:numElem

    elemNode = elem(e,:);
    coord = node(elemNode,:);

    Ke = Q4ElementStiffness(coord, D, b);

    dof = zeros(8,1);
    for i = 1:4
        dof(2*i-1) = 2*elemNode(i)-1;
        dof(2*i)   = 2*elemNode(i);
    end

    K(dof,dof) = K(dof,dof) + Ke;

end

leftNodes = find(abs(node(:,1)) < 1e-12);
[~, idxSort] = sort(node(leftNodes,2));
leftNodes = leftNodes(idxSort);

for i = 1:length(leftNodes)-1

    n1 = leftNodes(i);
    n2 = leftNodes(i+1);

    edgeLength = abs(node(n2,2) - node(n1,2));

    F(2*n1) = F(2*n1) - P * edgeLength/(2*(2*a));
    F(2*n2) = F(2*n2) - P * edgeLength/(2*(2*a));

end

fixedNodes = find(abs(node(:,1)-L) < 1e-12);
```

```
        fixedDOF = zeros(2*length(fixedNodes),1);
        for i = 1:length(fixedNodes)
            fixedDOF(2*i-1) = 2*fixedNodes(i)-1;
            fixedDOF(2*i)   = 2*fixedNodes(i);
        end

        allDOF  = (1:numDOF)';
        freeDOF = setdiff(allDOF, fixedDOF);

        U = zeros(numDOF,1);
        U(freeDOF) = K(freeDOF,freeDOF) \ F(freeDOF);

        [x_strain, y_strain, eps_num, sig_num] =
computeElementStrainStressAtCenter(node, elem, U, D);

        all_x_strain{m} = x_strain;
        all_y_strain{m} = y_strain;
        all_eps_num{m}  = eps_num;
        all_sig_num{m}  = sig_num;

end

x_ref   = all_x_strain{end};
y_ref   = all_y_strain{end};
eps_ref = all_eps_num{end};
sig_ref = all_sig_num{end};

F_eps_ref = scatteredInterpolant(x_ref, y_ref, eps_ref, 'linear', 'nearest');
F_sig_ref = scatteredInterpolant(x_ref, y_ref, sig_ref, 'linear', 'nearest');

for m = 1:length(nx_list)

        x_strain = all_x_strain{m};
        y_strain = all_y_strain{m};
        eps_num  = all_eps_num{m};
        sig_num  = all_sig_num{m};

        eps_ref_interp = F_eps_ref(x_strain, y_strain);
```

```
sig_ref_interp = F_sig_ref(x_strain, y_strain);

eps_error_ref = eps_num - eps_ref_interp;
sig_error_ref = sig_num - sig_ref_interp;

rms_strain_error_ref_list(m) = sqrt(mean(eps_error_ref.^2));
rms_stress_error_ref_list(m) = sqrt(mean(sig_error_ref.^2));

if m == length(nx_list)
    rms_strain_error_ref_list(m) = NaN;
    rms_stress_error_ref_list(m) = NaN;
end

fprintf('\n=========================================\n');
fprintf('nx = %d, ny = %d\n', nx_list(m), ny_list(m));
fprintf('hx = %.12e\n', hx_list(m));
fprintf('hy = %.12e\n', hy_list(m));
fprintf('h  = %.12e\n', h_list(m));

if m ~= length(nx_list)
    fprintf('RMS Strain Error compared with finest mesh = %.12e\n', ...
        rms_strain_error_ref_list(m));
    fprintf('RMS Stress Error compared with finest mesh = %.12e\n', ...
        rms_stress_error_ref_list(m));
else
    fprintf('RMS Strain Error compared with finest mesh = NaN\n');
    fprintf('RMS Stress Error compared with finest mesh = NaN\n');
end

fprintf('=========================================\n');

strain_stress_table = table(x_strain, y_strain, ...
    eps_num, eps_ref_interp, abs(eps_error_ref), ...
    sig_num, sig_ref_interp, abs(sig_error_ref), ...
    'VariableNames', {'x', 'y', ...
    'Numerical_Axial_Strain', ...
    'Finest_Mesh_Reference_Strain', ...
    'Absolute_Strain_Error_Reference', ...
```

```
            'Numerical_Axial_Stress', ...
            'Finest_Mesh_Reference_Stress', ...
            'Absolute_Stress_Error_Reference'});

        disp(strain_stress_table);

end

result_table = table(nx_list', ny_list', hx_list', hy_list', h_list', ...
        rms_strain_error_ref_list', rms_stress_error_ref_list', ...
        'VariableNames', {'nx', 'ny', 'hx', 'hy', 'h', ...
        'RMS_Strain_Error_Finest_Mesh_Reference', ...
        'RMS_Stress_Error_Finest_Mesh_Reference'});

disp(result_table);

figure;
loglog(h_list(1:end-1), rms_strain_error_ref_list(1:end-1), '-o', ...
        'LineWidth', 1.8, 'MarkerSize', 7);
grid on;
set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Strain Error');
title('Log-Log Plot of RMS Strain Error vs h using Finest-Mesh FEM Reference');

figure;
loglog(h_list(1:end-1), rms_stress_error_ref_list(1:end-1), '-o', ...
        'LineWidth', 1.8, 'MarkerSize', 7);
grid on;
set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Stress Error');
title('Log-Log Plot of RMS Stress Error vs h using Finest-Mesh FEM Reference');

figure;
hold on; grid on;
```

```
loglog(h_list(1:end-1), rms_strain_error_ref_list(1:end-1), '-o', ...
    'LineWidth', 1.8, 'MarkerSize', 7, ...
    'DisplayName', 'RMS Strain Error');

loglog(h_list(1:end-1), rms_stress_error_ref_list(1:end-1), '-s', ...
    'LineWidth', 1.8, 'MarkerSize', 7, ...
    'DisplayName', 'RMS Stress Error');

set(gca, 'XScale', 'log');
set(gca, 'YScale', 'log');
xlabel('h');
ylabel('RMS Error');
title('Log-Log Plot of RMS Strain and Stress Error vs h');
legend('show', 'Location', 'best');

p_strain = polyfit(log(h_list(1:end-1)), log(rms_strain_error_ref_list(1:end-1)), 1);
p_stress = polyfit(log(h_list(1:end-1)), log(rms_stress_error_ref_list(1:end-1)), 1);

fprintf('\n========================================\n');
fprintf('Estimated strain convergence order = %.12e\n', p_strain(1));
fprintf('Estimated stress convergence order = %.12e\n', p_stress(1));
fprintf('========================================\n');



function [node, elem] = generateMesh(L, a, nx, ny)

    x = linspace(0, L, nx+1);
    y = linspace(-a, a, ny+1);

    node = zeros((nx+1)*(ny+1), 2);

    count = 1;
    for j = 1:ny+1
        for i = 1:nx+1
            node(count,:) = [x(i), y(j)];
            count = count + 1;
        end
```

```
    end

    elem = zeros(nx*ny, 4);

    count = 1;
    for j = 1:ny
        for i = 1:nx

            n1 = (j-1)*(nx+1) + i;
            n2 = n1 + 1;
            n3 = n2 + (nx+1);
            n4 = n1 + (nx+1);

            elem(count,:) = [n1 n2 n3 n4];
            count = count + 1;

        end
    end

end


function Ke = Q4ElementStiffness(coord, D, thickness)

    gaussPoint = [-1/sqrt(3), 1/sqrt(3)];
    Ke = zeros(8,8);

    for i = 1:2
        for j = 1:2

            xi  = gaussPoint(i);
            eta = gaussPoint(j);

            dN_dxi  = 1/4 * [-(1-eta),  (1-eta),  (1+eta), -(1+eta)];
            dN_deta = 1/4 * [-(1-xi), -(1+xi),  (1+xi),  (1-xi)];

            J = [dN_dxi; dN_deta] * coord;
            detJ = det(J);
```

```
            invJ = inv(J);

            dN = invJ * [dN_dxi; dN_deta];

            B = zeros(3,8);

            for k = 1:4
                B(1,2*k-1) = dN(1,k);
                B(2,2*k)   = dN(2,k);
                B(3,2*k-1) = dN(2,k);
                B(3,2*k)   = dN(1,k);
            end

            Ke = Ke + B' * D * B * thickness * detJ;

        end
    end

end


function [x_strain, y_strain, eps_num, sig_num] =
computeElementStrainStressAtCenter(node, elem, U, D)

    numElem = size(elem,1);

    x_strain = zeros(numElem,1);
    y_strain = zeros(numElem,1);
    eps_num  = zeros(numElem,1);
    sig_num  = zeros(numElem,1);

    xi = 0;
    eta = 0;

    dN_dxi   = 1/4 * [-(1-eta),  (1-eta),  (1+eta), -(1+eta)];
    dN_deta = 1/4 * [-(1-xi), -(1+xi),  (1+xi),  (1-xi)];

    for e = 1:numElem
```

```
            elemNode = elem(e,:);
            coord = node(elemNode,:);

            J = [dN_dxi; dN_deta] * coord;
            invJ = inv(J);

            dN = invJ * [dN_dxi; dN_deta];

            B = zeros(3,8);

            for k = 1:4
                B(1,2*k-1) = dN(1,k);
                B(2,2*k)   = dN(2,k);
                B(3,2*k-1) = dN(2,k);
                B(3,2*k)   = dN(1,k);
            end

            dof = zeros(8,1);
            for k = 1:4
                dof(2*k-1) = 2*elemNode(k)-1;
                dof(2*k)   = 2*elemNode(k);
            end

            Ue = U(dof);

            strain = B * Ue;
            stress = D * strain;

            x_strain(e) = mean(coord(:,1));
            y_strain(e) = mean(coord(:,2));
            eps_num(e)  = strain(1);
            sig_num(e)  = stress(1);

        end

    end
```